



UNIVERSITY OF ASIA PACIFIC
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Third Year, Second Semester

Course Name: Software Engineering Lab

Course Code: 314

Date of Submission: 07/09/2026

SkillBridge Requirement Specification, Architecture, Design
**SkillBridge — AI-Powered Career Recommendation and Skill
Gap Analysis Platform**

Submitted by:

Rafi Ahammed Rajon – 23201150 (Business Analyst / Project Manager)

Foujia Jahan Prome – 23201152 (Team Lead)

Sabbir Ahmed Shisir – 23201163 (Reporting Lead)

Mahinur E Jannat – 23201148 (QA Lead)

Submitted to:

Md. Ashraful Alam

Lecturer of CSE,

University of Asia Pacific

Contents

3.1 Software Requirements Specification (SRS) for Job Seekers Module	4
3.1.1 Introduction	4
3.1.1.1 Purpose	4
3.1.1.2. Scope	4
3.1.2 Overall Description	4
3.1.2.1 Product Perspective	4
3.1.2.2. User Classes and Characteristics	4
.	5
3.1.2.3. Operating Environment	5
3.1.3 Functional Requirements	5
3.1.4 Non-Functional Requirements	7
3.1.5 Constraints	7
3.1.6 Assumptions and Dependencies	7
3.1.7 Future Enhancements	8
3.1.8 Conclusion	8
3.1.9 Safety and Security Requirements	8
3.1.9.1 Access Requirements	8
3.1.9.2 Integrity Requirements	8
3.1.9.3 Privacy Requirements	9
3.2 Software Requirements Specification (SRS) for Recruiter / Employer Module	9
3.2.1 Introduction	9
3.2.1.1 Purpose	9
3.2.1.2 Scope	9
3.2.2 Overall Description	10
3.2.2.1 Product Perspective	10
3.2.2.2 User Classes and Characteristics	10
3.2.2.3 Operating Environment	10
3.2.3 Functional Requirements	10
3.2.4 Non-Functional Requirements	11
3.2.5 Constraints	11
3.2.6 Assumptions and Dependencies	12
3.2.7 Future Enhancements	12
3.2.8 Conclusion	12
3.2.9 Safety and Security Requirements	12
3.2.9.1 Access Requirements	12
3.2.9.2 Integrity Requirements	13
3.2.9.3 Privacy Requirements	13
3.3 Software Requirements Specification (SRS) for Admin / Authority Module	13
3.3.1 Introduction	13
3.3.1.1 Purpose	13
3.3.1.2 Scope	13

3.3.2 Overall Description	14
3.3.2.1 Product Perspective	14
3.3.2.2 User Classes and Characteristics	14
3.3.2.3 Operating Environment	14
3.3.3 Functional Requirements	15
3.3.4 Non-Functional Requirements	15
3.3.5 Constraints	16
3.3.6 Assumptions and Dependencies	16
3.3.7 Future Enhancements	16
3.3.8 Conclusion	17
3.3.9 Safety and Security Requirements	17
3.3.9.1 Access Requirements	17
3.3.9.2 Integrity Requirements	17
3.3.9.3 Privacy Requirements	17
3.4. Stakeholders for the System	18
3.4.1 Job Seekers (Primary Users)	18
3.4.2 Recruiters / Employers	18
3.4.3 Admin & Authority	19
.	19
Stakeholder Interaction Summary	19
3.5 Use Case Diagram	19
3.5.1 Job Seeker Use Cases	19
Use Case: Resume Upload, Skill Extraction & Company/Job Matching	21
3.5.2 Recruiter / Employer Use Cases	22
Use Case: Recruiter Registration, Candidate Search & Shortlisting	23
.	24
3.5.3 Admin / Authority Use Cases	24
Use Case: Admin Platform Data Management & Recruiter Verification	25
3.6. Activity Diagram	26
3.6.1. Job Seeker Activity Diagram	26
3.6.2. Recruiter / Employer Activity Diagram	26
3.6.3. Admin / Authority Activity Diagram	27
3.7. Swimlane Diagram	28
3.7.1. Job Seeker Swimlane Diagram	28
.	29
3.7.2. Recruiter / Employer Swimlane Diagram	29
.	29
.	30
3.7.3. Admin / Authority Swimlane Diagram	30
3.8. Class Diagram	30
3.9. Sequence Diagram	31
3.9.1. Job Seeker Sequence Diagram	31
3.9.2. Recruiter / Employer Sequence Diagram	32

3.9.3. Admin / Authority Sequence Diagram	33
	34
3.10. State Diagram	34
3.10.1. Job Seeker State Diagram	34
3.10.2. Recruiter / Employer State Diagram	35
3.10.3. Recruiter / Employer State Diagram	36
4. Architechure	37
4.1 Architectural Model/Style Used	37
4.1.2. Rationale for Choosing Architectural Model/Style:	40
4.2. Technology, Software, and Hardware Used:	40
5. Design	42
5.1 Component Level Design Following Pattern	42
Resume & Skill Matching Module	42
Recruiter / Employer Module	42
Admin Panel	43
5.2 Dataflow Diagram	43
Job Seeker DFD	43
Recruiter / Employer DFD	44
Admin / Authority DFD	44
5.2.1 Entity Relationship Diagram	45

3. Software Requirements Specification

3.1 Software Requirements Specification (SRS) for Job Seekers Module

3.1.1 Introduction

3.1.1.1 Purpose

The purpose of this document is to provide a detailed description of the requirements for the Job Seeker-facing side of SkillBridge. This is the module through which a job seeker — primarily a CSE student or fresh graduate — registers, uploads a resume, receives an AI-extracted skill profile, and browses company, job, and course recommendations built from that profile. It describes the functional and non-functional requirements, constraints, and assumptions that apply specifically to the Job Seeker's use of the platform.

3.1.1.2. Scope

The Job Seeker module covers everything a registered job seeker can do end to end: creating an account, uploading a resume in PDF or DOCX format, having that resume automatically parsed and its skills extracted and normalized, viewing ranked company and job recommendations with a readiness score, viewing an explainable skill-gap breakdown against a target company or job, following course recommendations mapped to each missing skill, and controlling whether their extracted profile is visible to recruiters. Company, job, and course reference data that the Job Seeker browses is entered and maintained by the Admin (Section 3.3) and is treated as given input to this module rather than something the Job Seeker creates. The module is designed based on survey data from 84 student/job-seeker respondents, of whom 90% want a system that automatically analyzes their resume and 93% are likely or very likely to use the platform if it becomes available.

3.1.2 Overall Description

3.1.2.1 Product Perspective

The Job Seeker module is one of three actor-facing modules on the SkillBridge platform, alongside the Recruiter/Employer module (Section 3.2) and the Admin/Authority module (Section 3.3). It relies on the same centralized PostgreSQL database, accessed through the SQLAlchemy ORM, that the other modules read from and write to. The system is a web-based application built with a FastAPI (Python) backend and a React frontend, accessible through modern browsers on both desktop and mobile devices. Resume parsing uses pdfplumber (PDF) and python-docx (DOCX); the extracted text is passed through an NLP/ML skill-extraction layer, initially backed by the DeepSeek API, kept as a separate, replaceable component so that every recommendation the Job Seeker sees remains explainable back to specific matched and missing skills.

3.1.2.2. User Classes and Characteristics

User Class	Description
Job Seeker (Student)	A currently enrolled university student — the largest surveyed group (87% of respondents), predominantly CSE (45%) — who uploads a resume to get an early read on their skill profile and readiness for target roles.
Job Seeker (Fresh Graduate)	A recent graduate (8% of respondents) actively applying to roles, who relies on the platform's company/job recommendations and skill-gap results to prioritize which courses to take before applying.
Admin / Authority	Not a direct user of this module, but the secondary actor whose maintenance of company, job, skill, and course reference data (Section 3.3) is a precondition for every recommendation a Job Seeker sees.

3.1.2.3. Operating Environment

The Job Seeker module is a web-based application built with the FastAPI (Python) framework on the backend, PostgreSQL for relational data storage, and React for the frontend. It is accessible through modern browsers (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari) on both desktop and mobile devices, and the interface is responsive to accommodate the large share of first-time job seekers who may access the platform from a smartphone.

3.1.3 Functional Requirements

Registration & Authentication

- The system shall allow a job seeker to register and log in using an email and password, or Google OAuth.
- The system shall provide a password recovery mechanism through the registered email.
- The system shall not allow a job seeker to view any other job seeker's resume, profile, or contact details.

Resume Upload & Parsing

- The system shall allow a job seeker to upload a resume in PDF or DOCX format only, and to replace a previously uploaded resume.
- The system shall validate the uploaded file's format and size before accepting it, and reject invalid files with a clear error.
- The system shall extract raw text from the uploaded file (pdfplumber for PDF, python-docx for DOCX).
- The system shall flag any resume it cannot parse for manual review rather than silently failing.

AI-Based Skill Extraction

- The system shall automatically identify skills, education, experience, and job titles from the resume text using NLP/ML, initially via the DeepSeek API.
- The system shall normalize extracted skills against an Admin-curated skill taxonomy so that variants (e.g. "JS", "JavaScript") are treated as the same skill.
- The system shall let the job seeker view their own extracted skill profile after analysis completes.

Company & Job Recommendation

- The system shall recommend targeted companies to the job seeker based on their extracted skill profile, each shown with a readiness score.
- The system shall recommend specific, currently open job postings ranked by skill-match score, distinct from the broader company-fit recommendations.
- The system shall display job/company details including salary range, required skills, job responsibilities, work location, remote/hybrid option, and hiring requirements.
- The system shall notify the job seeker when a new job posting matches their extracted skill profile.

Skill Gap Analysis & Course Recommendation

- The system shall let the job seeker compare their existing skills against a target company's or job's requirements and view missing skills, ranked by how important that skill is to the target.
- The system shall recommend relevant online courses (e.g. Coursera, Udemy, YouTube, LinkedIn Learning, edX) to close each identified skill gap, using an Admin-curated skill-to-course mapping.
- The system shall recompute the job seeker's matches and skill gaps whenever they upload a new resume, or whenever an Admin updates a company, job, or course record.

User Dashboard

- The system shall provide the job seeker a personal dashboard summarizing their resume analysis, matched companies and jobs, skill gaps, and recommended courses.
- The dashboard shall display the top 3 matched skills and top 3 missing skills driving each recommendation score, so results are explainable.

Recruiter Visibility Consent

- The system shall let the job seeker explicitly opt in or opt out of recruiter visibility for their extracted skill profile, and shall record the consent decision with a timestamp.
- The system shall let the job seeker view a log of which recruiters viewed their extracted profile.

3.1.4 Non-Functional Requirements

Performance

- The system shall return resume analysis results within 10 seconds for resumes up to 5 MB under 50 concurrent users.
- The system shall let a first-time job seeker complete resume upload and view their first recommendation within 3 clicks/steps.

Security

- The system shall protect the job seeker's personal information and uploaded resume using AES-256 encryption at rest, TLS 1.2+ in transit, and bcrypt-hashed passwords.
- The system shall permanently delete a job seeker's resume file and extracted profile data within 30 days of an account-deletion request.

Usability

- The system shall provide a simple, user-friendly interface, given that a large share of job seekers are first-time users with low confidence in their skill match.
- Error messages (e.g. an unsupported resume format) shall be clear, descriptive, and provide guidance on corrective action.

Reliability

- The system shall correctly parse at least 95% of well-formatted, text-based PDF and DOCX resumes without extraction failure.
- The system shall maintain at least 99% monthly uptime, excluding scheduled maintenance windows announced at least 24 hours in advance.

3.1.5 Constraints

- The system shall only accept resumes in PDF or DOCX format from job seekers — no other file types.
- Skill extraction accuracy depends on the NLP/ML model, which may not be 100% accurate for all resume styles or formats.
- The company, job, and course data a job seeker is matched against is limited to what the Admin has manually entered — no automatic external API integration in the first version.

3.1.6 Assumptions and Dependencies

- It is assumed that job seekers will upload genuine, up-to-date resumes with accurate information.
- It is assumed that job seekers have access to a stable internet connection and a modern browser on a desktop or mobile device.
- The module depends on the Admin having populated enough company, job, and course reference data (Section 3.3) at launch to produce meaningful recommendations.

3.1.7 Future Enhancements

- The system could integrate course-provider APIs (Coursera, Udemy, edX, LinkedIn Learning) to pull live course catalogues and pricing instead of Admin-curated entries.
- The system could integrate job-board APIs (e.g. LinkedIn Jobs, Indeed) to broaden the pool of jobs a job seeker is matched against.
- The system could improve matching beyond simple skill overlap with semantic matching, related-skill inference, experience matching, and education matching, combined into an overall weighted score.
- The system could support a personalized, sequenced learning roadmap across multiple missing skills, rather than a flat list of course recommendations.

3.1.8 Conclusion

The Software Requirements Specification for the Job Seeker module defines the core functionality needed for a student or fresh graduate to go from an uploaded resume to an explainable set of company, job, and course recommendations. The requirements — automated resume parsing, AI-based skill extraction and normalization, ranked company/job matching with a readiness score, transparent skill-gap analysis, and mapped course recommendations — are directly supported by the requirement-gathering data: 90% of the 84 surveyed job seekers want a system that automatically analyzes their resume, and 93% are likely or very likely to use the platform if it becomes available. Keeping AI extraction, the structured profile, and the recommendation engine as separate components ensures every result shown to the job seeker can be traced back to specific matched and missing skills, which is essential for building trust with a user base that includes many first-time job seekers with low confidence in their own skill match.

3.1.9 Safety and Security Requirements

3.1.9.1 Access Requirements

- Job seekers should be able to register and log in using an email and password (or Google OAuth).
- Job seekers should be able to upload or replace their own resume and view their own extracted profile, recommendations, and skill gaps.
- Job seekers should be able to toggle recruiter-visibility consent and view a log of which recruiters viewed their profile.
- Job seekers should not be able to view any other job seeker's resume, profile, or contact details.

3.1.9.2 Integrity Requirements

- The system shall implement data validation checks to ensure only valid data is entered.

- Each candidate search result and shortlist entry shall be linked to a specific job posting and a specific candidate skill profile to ensure traceability.
- Each skill-gap and recommendation record shall be linked to a specific resume version and a specific company/job record to ensure traceability.
- Matches shall be recomputed whenever the user uploads a new resume or an Admin updates a company, job, or course record, so recommendations stay current.

3.1.9.3 Privacy Requirements

- A job seeker's extracted profile shall be private by default and visible to recruiters only after the job seeker turns on the “Visible to Recruiters” toggle, with the consent decision timestamped for audit purposes.
- Every recruiter access to a candidate's profile or contact details shall be logged so job seekers can see who viewed their data.

3.2 Software Requirements Specification (SRS) for Recruiter / Employer Module

3.2.1 Introduction

3.2.1.1 Purpose

The purpose of this document is to provide a detailed description of the requirements for the Recruiter/Employer-facing side of SkillBridge. This is the module through which a verified employer posts job requirements and searches consenting candidate profiles to support screening. It describes the functional and non-functional requirements, constraints, and assumptions that apply specifically to the Recruiter/Employer's use of the platform.

3.2.1.2 Scope

The Recruiter/Employer module covers registration with a work email and company name, Admin-mediated verification, job posting management (title, required skills, experience level, salary range), searching and filtering consenting candidate profiles by skill-match score, shortlisting candidates, and exchanging contact details only after a candidate accepts an interest notification. In the current implementation, the recruiter is not yet a directly-authenticated actor in the live database schema (Section 3.2.13, Class Diagram): a recruiter's company and job information reaches the platform through the Admin, who enters and maintains it in the companies and jobs tables. Recruiter self-service accounts, verification, candidate search, and consent-based visibility are documented here as the module's functional requirements and are the next increment of work, building on the Admin-mediated relationship already in place. The module is designed based on survey data from 5 recruiter/employer respondents, 80% of whom work in the Software/IT sector and 60% of whom said they would consider using an AI-based resume analysis and recommendation system.

3.2.2 Overall Description

3.2.2.1 Product Perspective

The Recruiter/Employer module is one of three actor-facing modules on the SkillBridge platform, alongside the Job Seeker module (Section 3.1) and the Admin/Authority module (Section 3.3). It relies on the same centralized PostgreSQL database, accessed through the SQLAlchemy ORM, that the other modules read from and write to. The system is a web-based application built with a FastAPI (Python) backend and a React frontend. A recruiter's core relationship is with the Admin, who verifies new recruiter accounts and, in the current schema, maintains the company and job records a recruiter's postings ultimately populate.

3.2.2.2 User Classes and Characteristics

User Class	Description
Recruiter / Employer	A verified employer-side account (HR executive, team lead, or hiring manager) that posts job requirements and searches consenting candidate profiles by skill-match score to support screening. Registration requires a work email and company name, and is manually approved by an Admin before the account is activated.
Job Seeker	Not a direct user of this module, but the secondary actor whose consent (the “Visible to Recruiters” toggle) determines which candidate profiles a recruiter can see, and who receives and responds to interest notifications.
Admin / Authority	The secondary actor who reviews and approves or rejects pending recruiter accounts, and who maintains the company and job reference data (Section 3.3) that a recruiter's postings and searches depend on.

3.2.2.3 Operating Environment

The Recruiter/Employer module is a web-based application built with the FastAPI (Python) framework on the backend, PostgreSQL for relational data storage, and React for the frontend. It is accessible through modern browsers (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari) on both desktop and mobile devices.

3.2.3 Functional Requirements

Recruiter Registration & Verification

- The system shall allow a Recruiter/Employer to register with a work email and company name; the account shall be created in a “Pending Verification” state.
- The system shall check the recruiter's email domain against the stated company name and queue the account for Admin review.
- The system shall require an Admin to manually approve or reject a Pending recruiter account before it can search candidates or post jobs.

Recruiter Candidate Search & Screening

- The system shall allow a verified recruiter to view candidate skill profiles extracted from resumes to support screening.
- The system shall surface candidate technical skills, work experience, certifications, soft skills, portfolio, and project details in the recruiter view.
- The system shall let a job seeker explicitly opt in or opt out of recruiter visibility for their extracted skill profile, and shall record the consent decision with a timestamp.
- The system shall let a verified recruiter search and filter consenting candidate profiles by skill-match score, without exposing the candidate's contact details or original resume file.
- The system shall let a recruiter shortlist a candidate and send an interest notification; the candidate's contact information shall be shared with the recruiter only after the candidate accepts.

Recruiter Job Posting Management

- The system shall let a verified recruiter create, edit, and close their own job postings (title, required skills, experience level, salary range), which feed both candidate search and job/company recommendations shown to job seekers.

3.2.4 Non-Functional Requirements

Performance

- The system shall support at least 100 concurrent users, including mixed job-seeker and recruiter sessions, with an average API response time under 2 seconds and a request failure rate below 1%.

Security

- The system shall enforce role-based access control (RBAC) restricting a recruiter's access to only the candidate data fields defined in Section 3.2.16, and shall protect all data in transit with TLS 1.2+ and at rest with AES-256 encryption.

Usability

- The recruiter's candidate-search results shall clearly surface the skill-match score and matched skills driving each result, so screening decisions are explainable.

Reliability

- The system shall maintain at least 99% monthly uptime, excluding scheduled maintenance windows announced at least 24 hours in advance, so job postings and candidate searches remain available.

3.2.5 Constraints

- A recruiter account cannot search candidates or post jobs until an Admin has manually approved it — there is no self-service activation.

- In the current schema there is no recruiter account table and no column linking a user to the company_id on jobs, so a recruiter's company and job information is entered and maintained by the Admin rather than by the recruiter directly.
- Email-domain verification against the stated company name is a heuristic check, not a definitive identity verification.

3.2.6 Assumptions and Dependencies

- It is assumed that a recruiter registers with a genuine work email that matches their stated company's domain.
- It is assumed that at least one job seeker has opted in to recruiter visibility, or has applied directly to a posting, before a recruiter's candidate search can return results.
- The module depends on the Admin's timely review of pending recruiter accounts (Section 3.3) to avoid activation delays.

3.2.7 Future Enhancements

- Automated company-domain verification against a business registry could replace the current manual Admin check.
- In-platform messaging between recruiter and candidate could be introduced once a candidate accepts an interest notification.
- Payment gateway APIs could be integrated if a premium recruiter tier is introduced.

3.2.8 Conclusion

The Software Requirements Specification for the Recruiter/Employer module defines the requirements needed for a verified employer to post job requirements and screen candidates without accessing data a job seeker has not consented to share. The requirements — Admin-mediated registration and verification, consent-gated candidate search, and job posting management — are directly supported by the requirement-gathering data: of 5 surveyed recruiters, 80% work in the Software/IT sector, 60% already use an Applicant Tracking System, and 60% said they would consider using an AI-based resume analysis and recommendation system. Keeping candidate visibility strictly consent-based, and contact-detail exchange gated on candidate acceptance, ensures the module supports recruiter screening needs without compromising job seeker privacy.

3.2.9 Safety and Security Requirements

3.2.9.1 Access Requirements

- Recruiters should be able to register, and search or post jobs only once their account is approved by an Admin.
- Recruiters should be able to view the structured fields (skills, experience, certifications, soft skills, portfolio) of consenting candidates only.

- Recruiters should not be able to view a candidate's original resume file or contact details before the candidate accepts an interest notification.
- Recruiters should not be able to view another recruiter's shortlist.

3.2.9.2 Integrity Requirements

- The system shall use database constraints (primary keys, foreign keys, unique constraints) to prevent inconsistent or incomplete recruiter, job, and company data.
- Each candidate search result and shortlist entry shall be linked to a specific job posting and a specific candidate skill profile to ensure traceability.
- Matches surfaced to a recruiter shall be recomputed whenever a job posting is edited or a candidate's resume is updated, so search results stay current.

3.2.9.3 Privacy Requirements

- A job seeker's extracted profile shall be private by default and visible to recruiters only after the job seeker turns on the “Visible to Recruiters” toggle, with the consent decision timestamped for audit purposes.
- Recruiters shall never see a candidate's raw resume file or contact details until the candidate accepts an interest notification.
- Every recruiter access to a candidate's profile or contact details shall be logged so job seekers can see who viewed their data.

3.3 Software Requirements Specification (SRS) for Admin / Authority Module

3.3.1 Introduction

3.3.1.1 Purpose

The purpose of this document is to provide a detailed description of the requirements for the Admin/Authority side of SkillBridge. This is the module through which Career Development Club members and project team members approve recruiter accounts, maintain the company, job, skill, and course reference data that the Job Seeker and Recruiter modules depend on, and monitor the platform for policy compliance and data-privacy issues.

3.3.1.2 Scope

The Admin/Authority module covers reviewing and approving or rejecting recruiter registration requests; creating, updating, and deleting company and job-role records; managing the skills taxonomy and job-skill requirements; managing courses and their mapping to skills; managing and monitoring user accounts, uploaded resumes, and extracted skills; and enforcing platform policies on content and user conduct. Every recommendation a Job Seeker or Recruiter sees ultimately depends on data this module maintains — nothing in Sections 3.1 or 3.2 is possible without it. The module is designed based on survey

data from 7 admin respondents: 100% said administrators should be able to manage registered users, 86% agreed administrators should be able to add, update, and delete company/job-role information and manage courses mapped to skills, and 71% rated protection of users' personal information and uploaded resumes at the highest importance level (5/5).

3.3.2 Overall Description

3.3.2.1 Product Perspective

The Admin/Authority module is the third of three actor-facing modules on the SkillBridge platform, alongside the Job Seeker module (Section 3.1) and the Recruiter/Employer module (Section 3.2). It is the only module with write access to the shared reference data — Companies, Jobs, Skills, and Courses — that the other two modules read from, and the only module that can approve or reject a Recruiter account. Like the other modules, it is built on the FastAPI (Python) backend, PostgreSQL database accessed through SQLAlchemy, and a React frontend, with every write attributable to a specific admin user through the `created_by_id` / `updated_by_id` audit columns present on each domain table.

3.3.2.2 User Classes and Characteristics

User Class	Description
Admin / Authority	A Career Development Club member or project team member who administers the platform — approving recruiters, maintaining reference data, and monitoring users. Of 7 surveyed admin representatives, all identified user-account management as a required capability.
Recruiter / Employer	Not a direct user of this module, but the actor whose account status (pending, approved, rejected, suspended) is entirely controlled by Admin decisions made here (Section 3.2.15).
Job Seeker	Not a direct user of this module, but the actor whose uploaded resumes, extracted skills, and account activity Admin can monitor for support or moderation purposes, within the access limits defined in Section 3.3.16.1.

3.3.2.3 Operating Environment

The Admin/Authority module is a web-based application built with the FastAPI (Python) framework on the backend, PostgreSQL for relational data storage, and React for the frontend admin panel. It is accessed through modern desktop browsers (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari); unlike the Job Seeker and Recruiter modules, mobile responsiveness is a lower priority given this module's administrative, desk-based usage pattern.

3.3.3 Functional Requirements

Recruiter Account Approval

- The system shall let the Admin view all pending recruiter registration requests, including the submitted work email and company name.
- The system shall let the Admin approve or reject a pending recruiter account, and notify the recruiter of the decision.
- The system shall let the Admin suspend or reactivate a previously approved recruiter account, and permanently remove an account when required.

Company & Job-Role Management

- The system shall let the Admin create, update, and delete company records.
- The system shall let the Admin create, update, and delete job-role (job posting) records, including salary range, required experience, and required education.

Skills Taxonomy & Job-Skill Requirements Management

- The system shall let the Admin create, update, and delete entries in the skills taxonomy that resume and job-skill extraction is normalized against.
- The system shall let the Admin define and update which skills are required (and which are preferred) for a given job-role record.

Course & Skill-to-Course Mapping Management

- The system shall let the Admin manage the course catalog and map individual courses to the specific skills they teach.

User Account Management & Monitoring

- The system shall let the Admin manage user accounts and monitor registered users, uploaded resumes, user activity, and extracted skills.
- The system shall let the Admin view platform-level statistics (e.g. registered users, resumes processed, active job postings).

Platform Policy Enforcement

- The system shall let the Admin enforce platform policies regarding content standards, user conduct, and data privacy, including reviewing a flagged resume or account and taking moderation action.

3.3.4 Non-Functional Requirements

Performance

- The admin panel shall support at least 100 concurrent platform users overall (mixed job-seeker, recruiter, and admin sessions) with an average API response time under 2 seconds.

Security

- Admin access shall be restricted to authorized platform staff only, enforced through role-based access control (RBAC) separate from Job Seeker and Recruiter roles.

- Every create, update, or delete performed by an Admin shall be attributable to that Admin's user account via the `created_by_id` / `updated_by_id` audit columns, and timestamped via `created_at` / `updated_at`.

Usability

- The admin panel shall present pending recruiter approvals, flagged content, and reference-data management in a single, consistent interface requiring no more training than a typical CRUD admin tool.

Reliability

- The system shall undergo regular audits to identify and address potential data-integrity issues, and shall use version control (Git/GitHub) to track changes to the codebase.
- Matches surfaced to Job Seekers and Recruiters shall be recomputed whenever the Admin updates a company, job, or course record, so recommendations stay current.

3.3.5 Constraints

- Company, job, and course data must be manually added and managed by the Admin in this version — no automatic external API integration.
- Admin access to an individual job seeker's resume file or personal contact details shall be limited to what is needed for support or moderation, not unrestricted browsing.
- Admin access itself shall be restricted to authorized platform staff only, distinct from the Job Seeker and Recruiter roles.

3.3.6 Assumptions and Dependencies

- It is assumed that Admin/Authority users are trusted Career Development Club members or project team members, not the general public.
- It is assumed that Admin users have access to a stable internet connection and a modern desktop browser.
- The Job Seeker and Recruiter modules (Sections 3.1 and 3.2) depend on the Admin populating and maintaining enough company, job, and course reference data to produce meaningful recommendations — this module is a precondition for both, not the reverse.

3.3.7 Future Enhancements

- Bulk import/export of company, job, and course records could be added to reduce manual data-entry effort for the Admin.
- Automated company-domain verification against a business registry could be explored to reduce manual recruiter-approval effort.
- A dedicated analytics dashboard could be added, extending the current platform-statistics view with skill-demand trends and recommendation-quality metrics over time.

- Role-based sub-admin permissions (e.g. a moderation-only role distinct from a data-management role) could be introduced as the admin team grows.

3.3.8 Conclusion

The Software Requirements Specification for the Admin/Authority module defines the requirements for the one actor with write access to SkillBridge's shared reference data and the sole authority to approve Recruiter accounts. The requirements — recruiter approval, company/job/skill/course management, user monitoring, and policy enforcement — are directly supported by the requirement-gathering data: 100% of the 7 surveyed admin representatives want user-account management capability, 86% want full CRUD control over company/job/course data, and 71% rated protection of users' personal data at the highest importance level. Because every Job Seeker recommendation (Section 3.1) and every Recruiter capability (Section 3.2) is ultimately expressed through data this module maintains, its requirements are a precondition for the rest of the system rather than an independent feature set.

3.3.9 Safety and Security Requirements

3.3.9.1 Access Requirements

- Admin should have access to approve or reject recruiter accounts.
- Admin should be able to manage companies, job roles, required skills, course-to-skill mappings, and user accounts.
- Admin access to an individual job seeker's resume file or personal contact details should be limited to what is needed for support or moderation.
- Admin access should be restricted to authorized platform staff only.

3.3.9.2 Integrity Requirements

- The system shall use database constraints (primary keys, foreign keys, unique constraints) to prevent inconsistent or incomplete company, job, skill, or course data entered by the Admin.
- Every Admin create, update, or delete action shall be attributable to that Admin's account and timestamped, via the `created_by_id` / `updated_by_id` and `created_at` / `updated_at` columns.
- Matches and recommendations shall be recomputed whenever the Admin updates a company, job, or course record, so results shown to Job Seekers and Recruiters stay current.

3.3.9.3 Privacy Requirements

- Admin access to job seekers' personal data and resumes shall be logged, consistent with the platform-wide principle that all recruiter and admin access to personal data is auditable.

- All data transmission to and from the admin panel shall occur over TLS 1.2+, and personal data shall be encrypted at rest using AES-256, the same as for Job Seekers and Recruiters.
- Admin should not access a job seeker's raw resume file or private contact details beyond what a specific support or moderation case requires.

3.4. Stakeholders for the System

Stakeholders are individuals or groups who have a vested interest in the SkillBridge platform, either as users, beneficiaries, or regulators. The following stakeholder groups were identified through the requirement-gathering process and will interact with or be impacted by the system.

3.4.1 Job Seekers (Primary Users)

Job seekers are the primary stakeholders and end-users of SkillBridge. Of the 84 survey respondents, 87% were students and 8% fresh graduates, with CSE the largest field of study (45%). 90% want a system that automatically analyzes their resume, and 93% are likely or very likely to use the platform if it becomes available.

Role in the System:

- Register and upload a resume in PDF or DOCX format for automatic analysis.
- View their extracted skill profile, recommended companies, and recommended open jobs.
- View their skill gaps relative to a target company or job, and follow the recommended courses.
- Opt in or out of recruiter visibility for their extracted profile, and view a log of which recruiters viewed their data.

3.4.2 Recruiters / Employers

Recruiters and employers provide job requirements, vacancies, salary information, and required skills, and use SkillBridge to support candidate screening. Of the 5 company respondents, 80% were from the Software/IT sector; 60% currently use an Applicant Tracking System (ATS) for resume screening, and identifying relevant skills together with incomplete resumes were the top-cited screening challenges. 60% said they would consider using an AI-based resume analysis/recommendation system.

Role in the System:

- Register with a work email and company name, and wait for Admin verification before the account is activated.
- Post and manage job requirements (title, required skills, experience level, salary range).
- Search and filter consenting candidate profiles by skill-match score.
- Shortlist candidates and exchange contact details only after the candidate accepts an interest notification.

3.4.3 Admin & Authority

Admin and Authority stakeholders are Career Development Club members and project team members who administer and manage the SkillBridge platform. Of the 7 admin respondents, 100% said administrators should be able to manage registered users, and 86% agreed administrators should be able to add, update, and delete company/job-role information and manage courses mapped to skills. 71% rated protection of users' personal information and uploaded resumes at the highest importance level (5/5).

Role in the System:

- Approve or reject recruiter registration requests.
- Create, update, and delete company, job-role, required-skill, and course-to-skill records.
- Manage user accounts and monitor registered users, uploaded resumes, user activity, and extracted skills.
- Enforce platform policies regarding content standards, user conduct, and data privacy.

Stakeholder Interaction Summary

Stakeholder	Interaction with SkillBridge
Job Seekers	Upload resume, view extracted skills, view company & job recommendations, view skill gaps, follow course recommendations, manage recruiter-visibility consent.
Recruiters / Employers	Register and get verified, post job requirements, search consenting candidate profiles by skill match, shortlist and message candidates.
Admin / Authority	Approve recruiters, manage companies/jobs/skills/courses, manage user accounts, monitor platform activity and privacy compliance.

3.5 Use Case Diagram

3.5.1 Job Seeker Use Cases

The Job Seeker is the platform's primary end user, and the only stakeholder with a directly-authenticated account in the current schema (the users table). A Job Seeker registers and logs in, uploads a resume, and the system stores the extracted skills in resume_skills. From there, the Job Seeker can browse the companies and jobs tables, view a job's required skills (via job_skills), compare that against their own extracted skills to see their skill gap, and view the courses mapped to any missing skill (via skills.id → courses.skill_id).

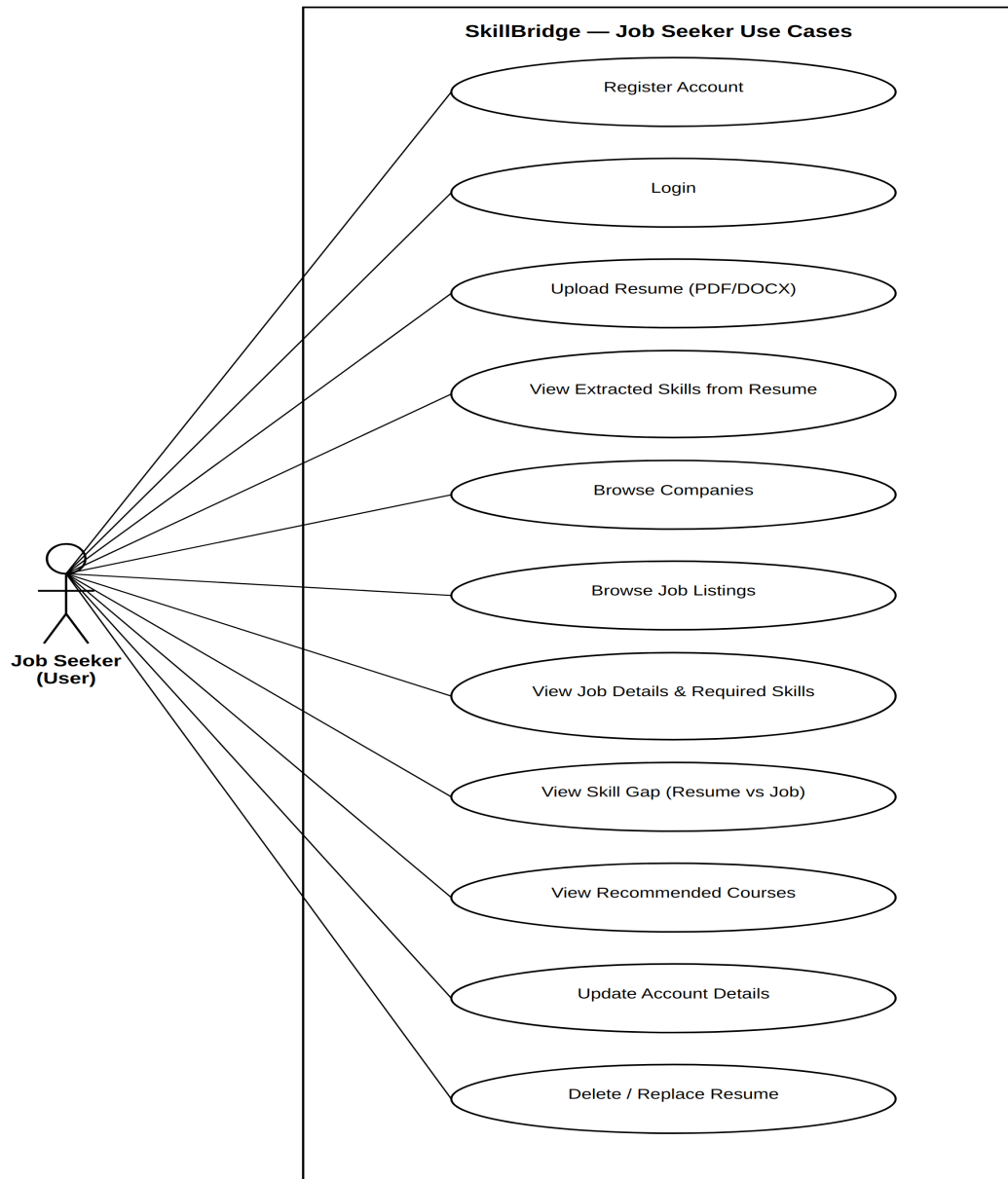


Figure 3.5.1: Use Case Diagram — Job Seeker

Use Case: Resume Upload, Skill Extraction & Company/Job Matching

Use Case: Resume Upload, Skill Extraction & Company/Job Matching

Iteration: 1

Primary Actors: Job Seeker

Goal in Context: To let a job seeker upload a resume and receive ranked company and job recommendations, with an explainable skill gap and matching course suggestions.

Preconditions:

- The user must be registered and logged in.
- The resume file must be in PDF or DOCX format and within the allowed size limit.
- The company/job/skill/course reference data must already be populated by the Admin.

Trigger: The job seeker selects “Upload Resume” and submits a PDF or DOCX file.

Scenario:

1. Upload: The job seeker selects and uploads a resume file.
2. Validation: The system checks the file format and size; invalid files are rejected with an error.
3. Extraction: The system extracts raw text from the resume (pdfplumber for PDF, python-docx for DOCX).
4. AI analysis: The system runs NLP/ML skill extraction on the text and normalizes results against the skill taxonomy.
5. Profile build: The system builds a weighted skill profile from the normalized skills.
6. Matching: The system compares the profile against companies and open job postings and computes readiness scores.
7. Gap analysis: The system identifies

missing skills for each recommended target, ranked by importance.

8. Course mapping: The system maps each missing skill to recommended courses.

9. Display: The dashboard shows matched companies, matched jobs, skill gaps, and course recommendations.

Exceptions:

1. An unsupported file type or oversized file triggers a validation error and the upload is rejected.
2. A resume the system cannot parse is flagged for manual review instead of failing silently.
3. No qualifying company or job meets the minimum relevance threshold; the dashboard shows no matches with guidance on skills to build.

Priority: High priority, to be implemented as part of the core functionality.

When Available: First increment (MVP).

Frequency of Use: Frequent — every time a user uploads or replaces a resume.

Channel to Actor: Via a web application using an active internet connection.

Secondary Actors: Admin (maintains company, job, skill, and course reference data used by the matching engine).

Channels to Secondary Actors: Accessed through the admin panel using an internet connection.

Open Issues:

1. Live job-board API integration can be introduced in a later version.
2. Semantic/embedding-based matching can replace or augment keyword-based skill matching

3.5.2 Recruiter / Employer Use Cases

The Recruiter/Employer is not a directly-authenticated actor in the current schema — there is no recruiter account table and no column linking a user to the company_id on jobs. In the implemented system, a recruiter's company and job information is supplied to the Admin, who enters and maintains it in the companies and jobs tables (Section 5, Constraints). The recruiter's use cases below reflect this Admin-mediated relationship, together with the public, read-only interactions (viewing a published listing, viewing aggregate skill-demand trends drawn from skills and resume_skills) that do not require their own login.

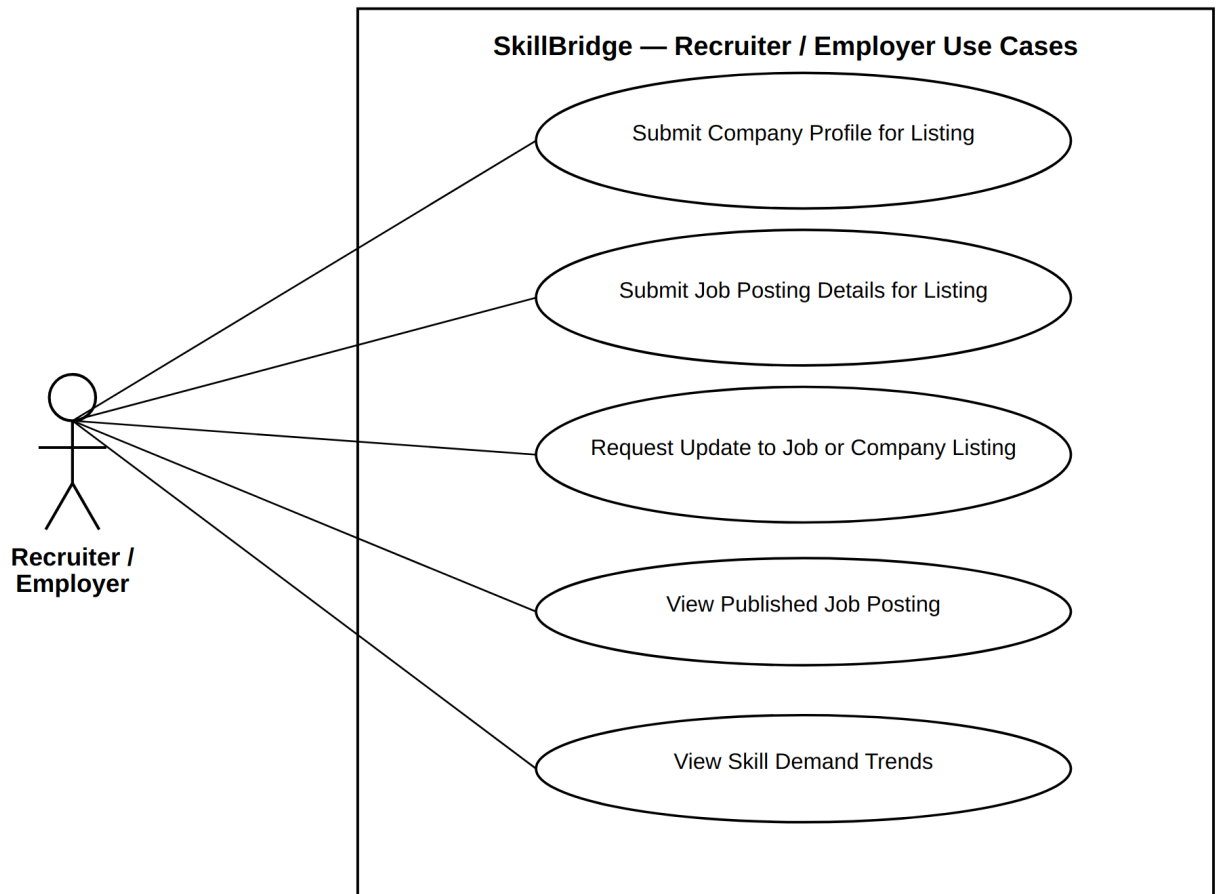


Figure 3.5.2: Use Case Diagram — Recruiter / Employer

Use Case: Recruiter Registration, Candidate Search & Shortlisting

Use Case: Recruiter Registration, Candidate Search & Shortlisting

Iteration: 2

Primary Actors: Recruiter / Employer

Goal in Context: To let a verified recruiter search consenting candidate profiles by skill match and shortlist candidates without accessing data the candidate has not consented to share.

Preconditions:

- The recruiter must have a valid work email and company name.
- The admin verification workflow must be in place.
- At least one job seeker must have opted in to recruiter visibility, or applied directly to the recruiter's posting.

Trigger: A recruiter registers on SkillBridge and selects “Register as Recruiter”, or a verified recruiter opens “Search Candidates”.

Scenario:

1. Registration: The recruiter submits their name, work email, and company name.
2. Pending state: The system creates the account in a “Pending Verification” state and checks the email domain against the company name.
3. Admin review: An Admin reviews the pending account and approves or rejects it.
4. Activation: On approval, the system activates the account and the recruiter can log in.
5. Posting: The recruiter creates a job posting with title, required skills, experience level, and salary range.
6. Search: The recruiter searches and filters consenting candidate profiles by skill-match score against the posting.
7. Shortlist: The recruiter shortlists a matching candidate; the system sends the can-

didate an interest notification.

8. Contact exchange: If the candidate accepts, contact details are exchanged; otherwise no contact information is shared.

Exceptions:

1. An incomplete registration form triggers a validation error.
2. The Admin rejects the account if the email domain cannot be verified against the company name; the recruiter is notified.
3. The recruiter attempts to view a candidate who has not opted in and who has not applied to their posting; the system denies access.
4. The candidate declines or ignores the interest notification; no contact information is shared.

Priority: High — essential for the recruiter-facing side of the platform.

When Available: Second increment, after the core resume-to-recommendation flow (first increment).

Frequency of Use: Registration is occasional (one-time per recruiter); search and shortlisting are frequent.

Channel to Actor: Via a web application using an active internet connection.

Secondary Actors: Admin (verifies recruiter accounts), Job Seeker (grants or withholds visibility consent).

Channels to Secondary Actors: Admin — accessed through the admin panel; Job Seeker — accessed through the web application.

Open Issues:

1. Automated company-domain verification against a business registry can be explored.
2. In-platform messaging between recruiter and candidate after acceptance can be introduced.

3.5.3 Admin / Authority Use Cases

The Admin owns every reference table in the schema that is not directly tied to a Job Seeker's own account: companies, jobs, job_skills, skills, and courses. Because the schema has no recruiter-facing tables, the Admin is also the sole route by which company and job data enters the system. The Admin additionally has read access to the users, resumes, and resume_skills tables for account management, support, and platform reporting, consistent with the access rules in Section 16.

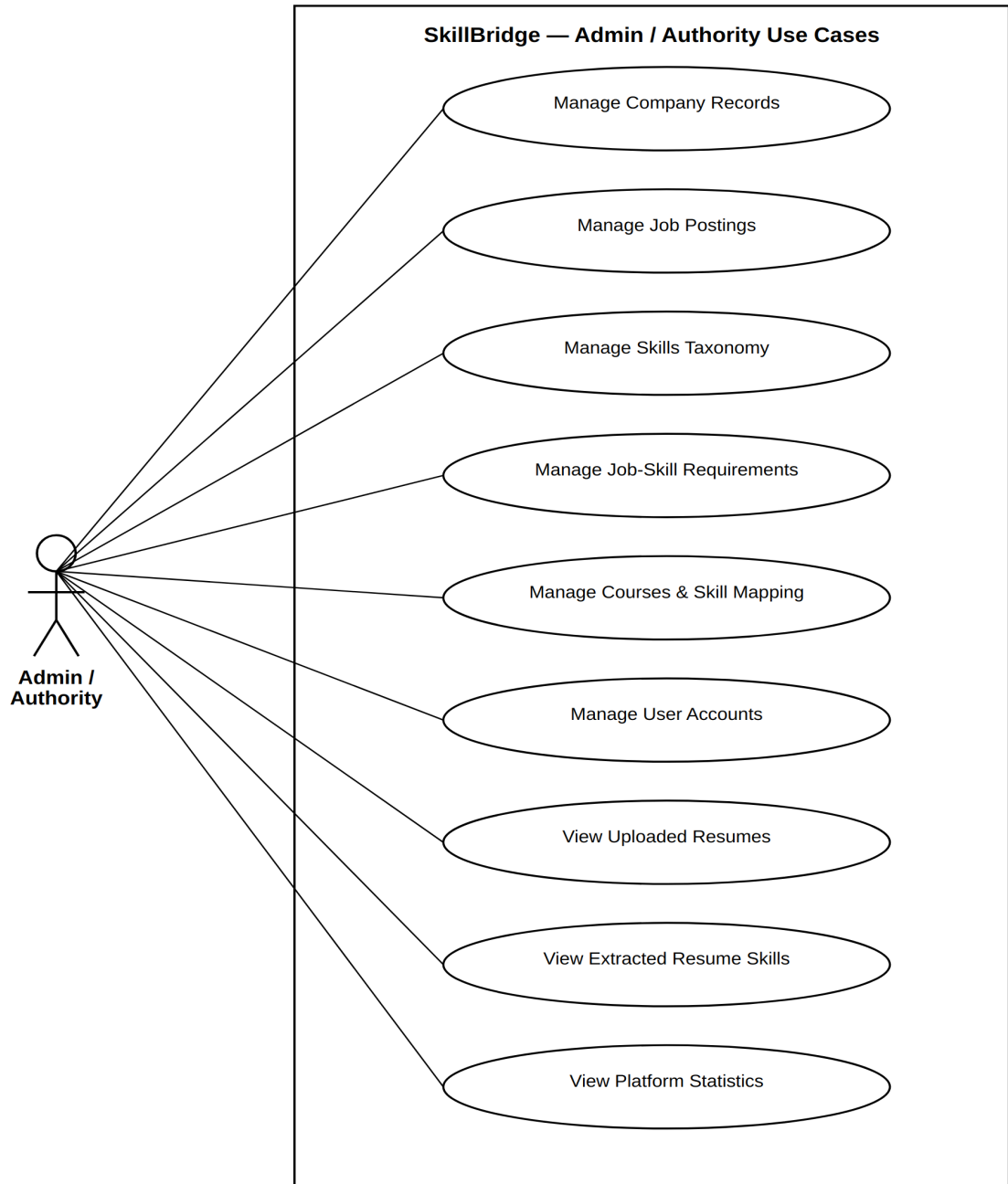


Figure 3.5.3: Use Case Diagram — Admin / Authority

Use Case: Admin Platform Data Management & Recruiter Verification

Use Case: Admin Platform Data Management & Recruiter Verification

Iteration: 2

Primary Actors: Admin / Authority

Goal in Context: To let an Admin verify recruiter accounts and maintain the company, job, skill, and course reference data that the matching engine depends on, while monitoring platform activity and privacy compliance.

Preconditions:

- The Admin account must already exist (Admin accounts are created only by an existing Admin).
- The Admin is authenticated with a role claim of admin.

Trigger: A recruiter account enters “Pending Verification”, or an Admin opens the admin panel to update reference data or review platform activity.

Scenario:

1. Review: The Admin opens the list of pending recruiter accounts and reviews the work email against the stated company name.
2. Decision: The Admin approves or rejects the recruiter account; the system notifies the recruiter of the outcome.
3. Data management: The Admin creates, updates, or deletes company records, job records, required skills, and course-to-skill mappings as the platform's dataset evolves.
4. Monitoring: The Admin reviews registered users, uploaded resumes, extracted skills, and user activity logs for moderation and support purposes.
5. Suspension: If needed, the Admin suspends an approved recruiter account; a suspended recruiter can later be reactivated.

Exceptions:

1. The recruiter's email domain does not match the stated company name; the Admin rejects the account with a reason.
2. An Admin attempts to view a job seeker's raw resume file or contact details outside of a support/moderation need; access is limited per the RBAC rules in Section 16.
3. A company, job, or course record fails validation (e.g. missing required fields); the system rejects the update.

Priority: Must — required for the recruiter-approval workflow and for keeping the matching engine's reference data current.

When Available: First increment, alongside the core resume-to-recommendation flow, since recruiter verification and reference data are prerequisites for the recruiter- and company-facing features.

Frequency of Use: Recruiter approvals are occasional; reference-data management and activity monitoring are ongoing, routine tasks.

Channel to Actor: Via the admin panel over a web application using an active internet connection.

Secondary Actors: Recruiter / Employer (the subject of the verification decision).

Channels to Secondary Actors: Recruiter — notified through the web application.

Open Issues:

1. Bulk import/export of company, job, and course records could be added to reduce manual data entry.
2. Automated anomaly detection on user activity logs could be explored for moderation support.

3.6. Activity Diagram

3.6.1. Job Seeker Activity Diagram

The activity diagram below traces the Job Seeker's path through the system: registration, resume upload and validation, text extraction, AI-based skill extraction, and on through matching, skill-gap analysis, and course recommendation to the personal dashboard. The two decision points reflect real failure modes — an invalid file format, or a resume the text-extraction step cannot parse — both handled explicitly rather than left to fail silently.

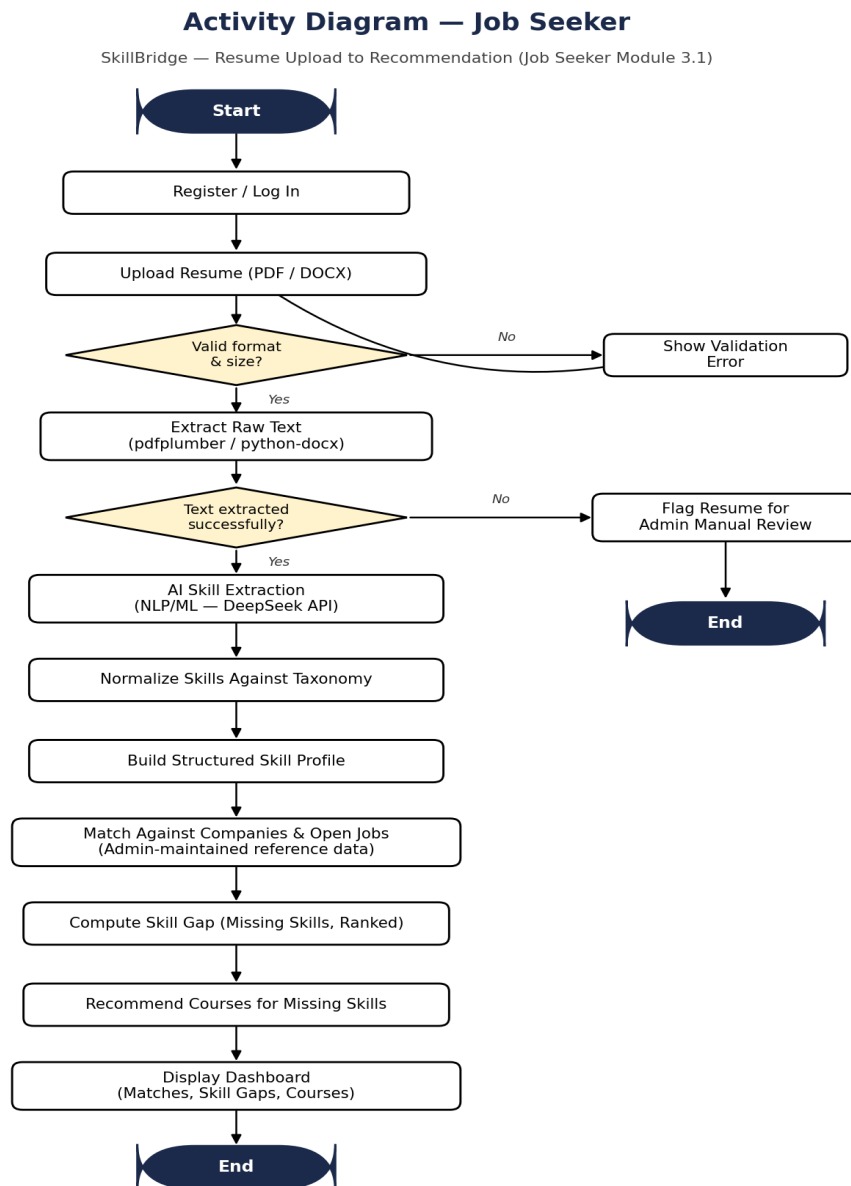


Figure 3.6.1: Activity Diagram — Job Seeker

3.6.2. Recruiter / Employer Activity Diagram

The activity diagram below traces a recruiter's path through the system: registration, Admin verification, job posting, consent-gated candidate search, shortlisting, and contact

exchange. Each of the four decision points reflects a real exception already identified for this module — an incomplete registration, an Admin rejection, an attempt to view a non-consenting candidate, and a candidate who declines an interest notification — each handled explicitly rather than left to fail silently.

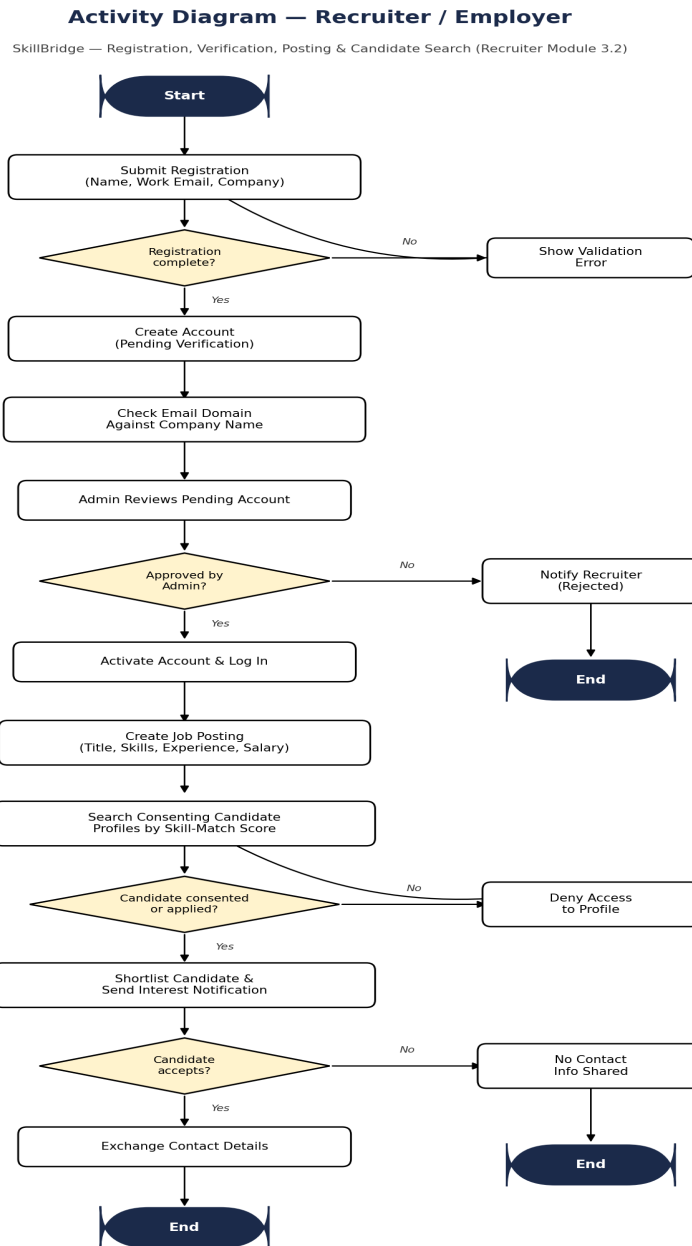


Figure 3.6.2: Activity Diagram — Recruiter / Employer

3.6.3. Admin / Authority Activity Diagram

The activity diagram below traces an Admin session end to end: reviewing any pending recruiter accounts first, then maintaining company, job, skill, and course reference data, then monitoring user accounts and handling any policy violation or support request that has come up, before checking platform statistics. The two decision points route the Admin toward recruiter approval or moderation action only when there is actually something pending — otherwise the flow continues straight through to routine data maintenance.

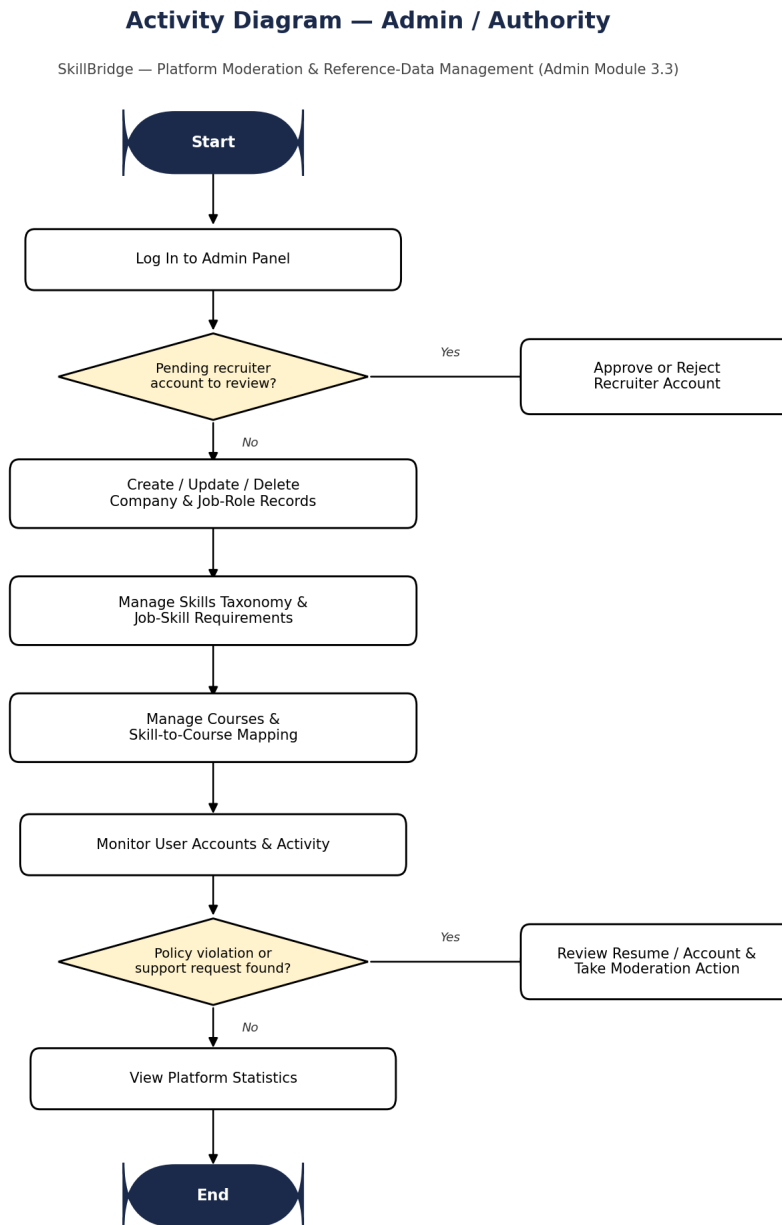


Figure 3.6.3: Activity Diagram — Admin / Authority

3.7. Swimlane Diagram

3.7.1. Job Seeker Swimlane Diagram

The swimlane diagram separates the same flow by responsibility across three actors: the Job Seeker (registration, upload, viewing results, consent), the System (validation, AI extraction, matching, skill-gap computation, rendering), and the Admin (maintaining the company, job, skill, and course reference data that the matching step depends on). Splitting the flow this way makes explicit that no recommendation reaches the Job Seeker without System processing in between, and no System processing produces a meaningful match without Admin-maintained reference data already in place.

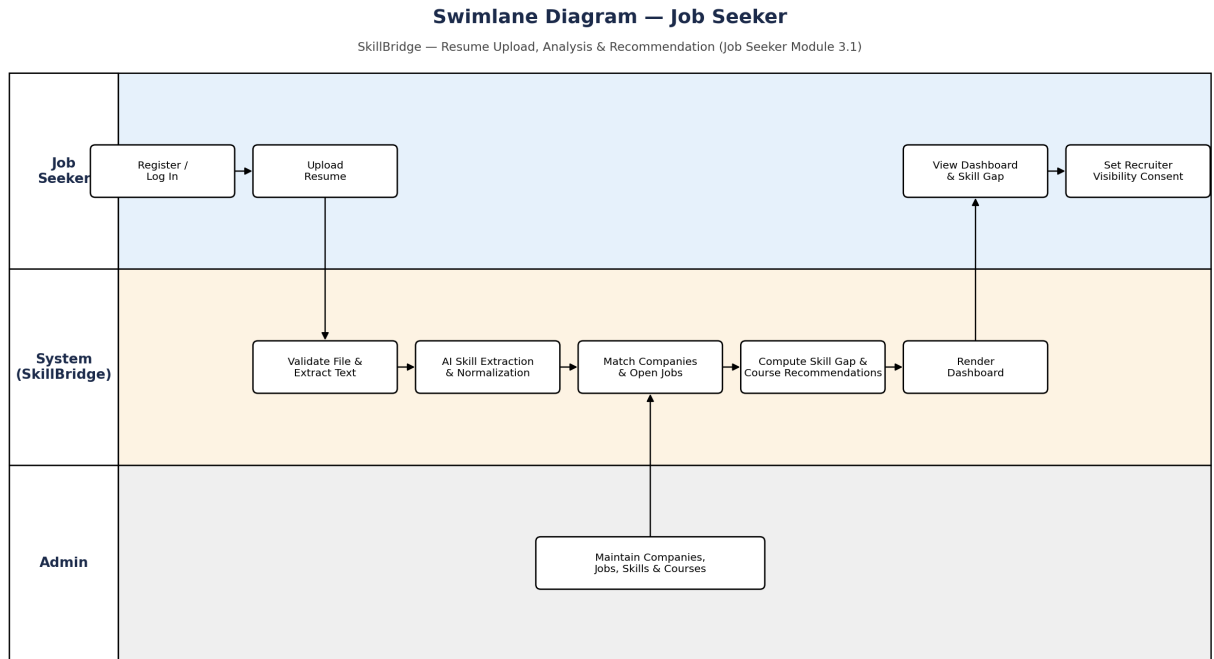


Figure 3.7.1: Swimlane Diagram — Job Seeker

3.7.2. Recruiter / Employer Swimlane Diagram

A swimlane diagram organizes the activities of a process into lanes, each representing a different participant, to show responsibilities and hand-offs clearly. The swimlane diagram below covers the full Recruiter/Employer workflow — registration, Admin verification, account activation, job posting, consent-gated candidate search, and candidate matching — across three lanes: Recruiter, System, and Admin.

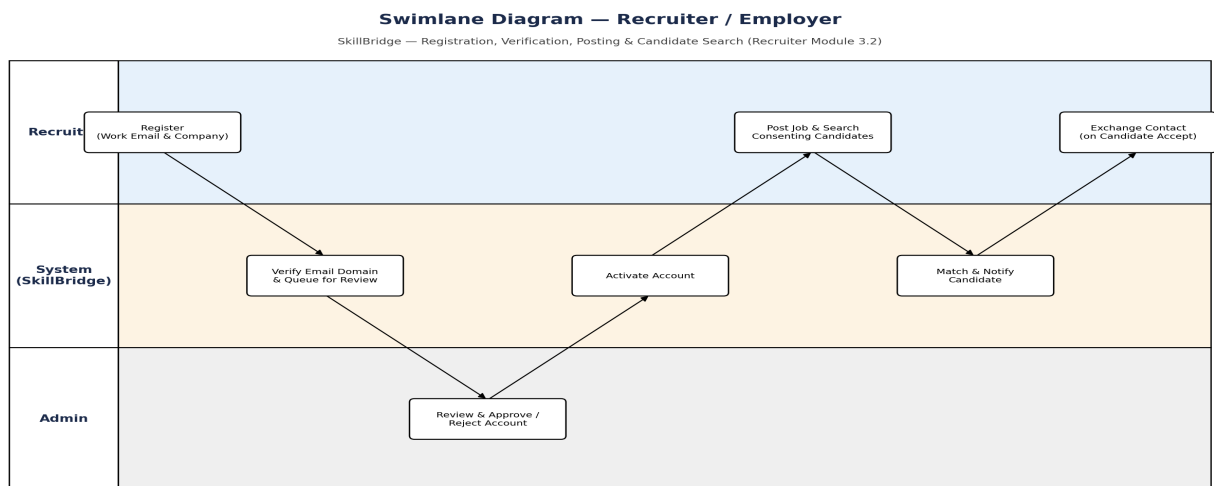


Figure 3.7.2: Swimlane Diagram — Recruiter / Employer

3.7.3. Admin / Authority Swimlane Diagram

The swimlane diagram separates responsibility across three participants: Recruiter/Job Seeker (the parties affected by Admin decisions), System, and Admin. It shows two connected cycles — recruiter account approval, and reference-data maintenance — both of which end with the System recomputing matches so that Job Seekers see the effect of the Admin's work on their own recommendations.

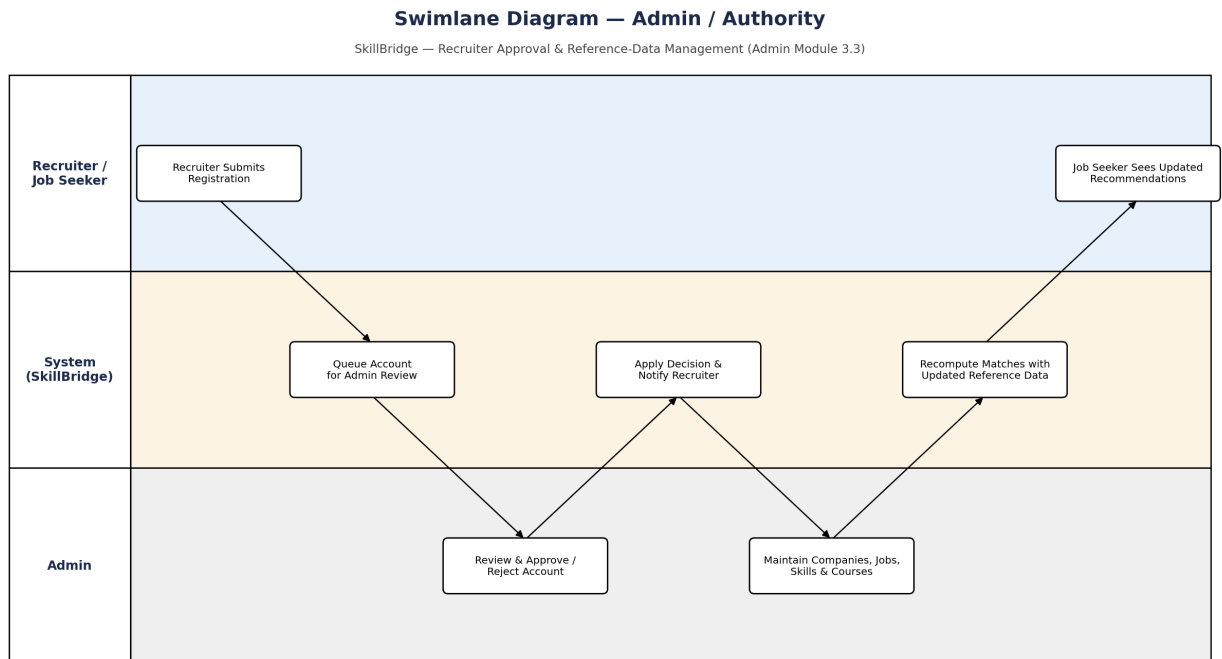


Figure 3.7.3: Swimlane Diagram — Admin / Authority

3.8. Class Diagram

A class diagram provides a static view of the system by illustrating its classes, attributes, and relationships. The class diagram for SkillBridge covers the core domain model: User (specialized into JobSeeker, Recruiter, and Admin), Resume, SkillProfile, Skill, Company, Job, Course, and Recommendation, and the associations that connect them.

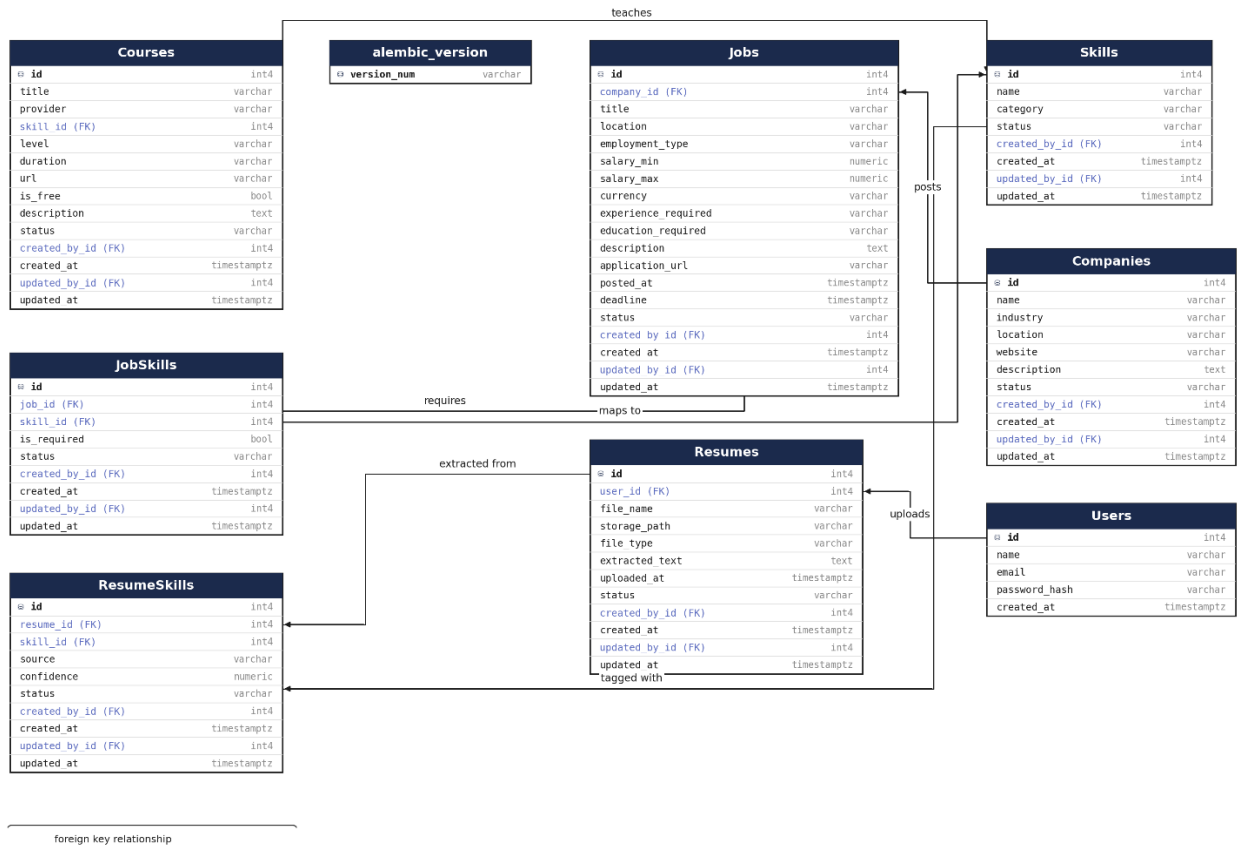


Figure 3.8: Class Diagram for SkillBridge

3.9. Sequence Diagram

3.9.1. Job Seeker Sequence Diagram

The sequence diagram below is specific to the Job Seeker module: uploading a resume, AI-based skill extraction, company and job matching, and skill-gap and course recommendation, ending with the dashboard display — distinct from the Recruiter sequence in Section 3.2.14 and the Admin sequence in Section 3.3.14, though all three ultimately act on the same underlying schema.

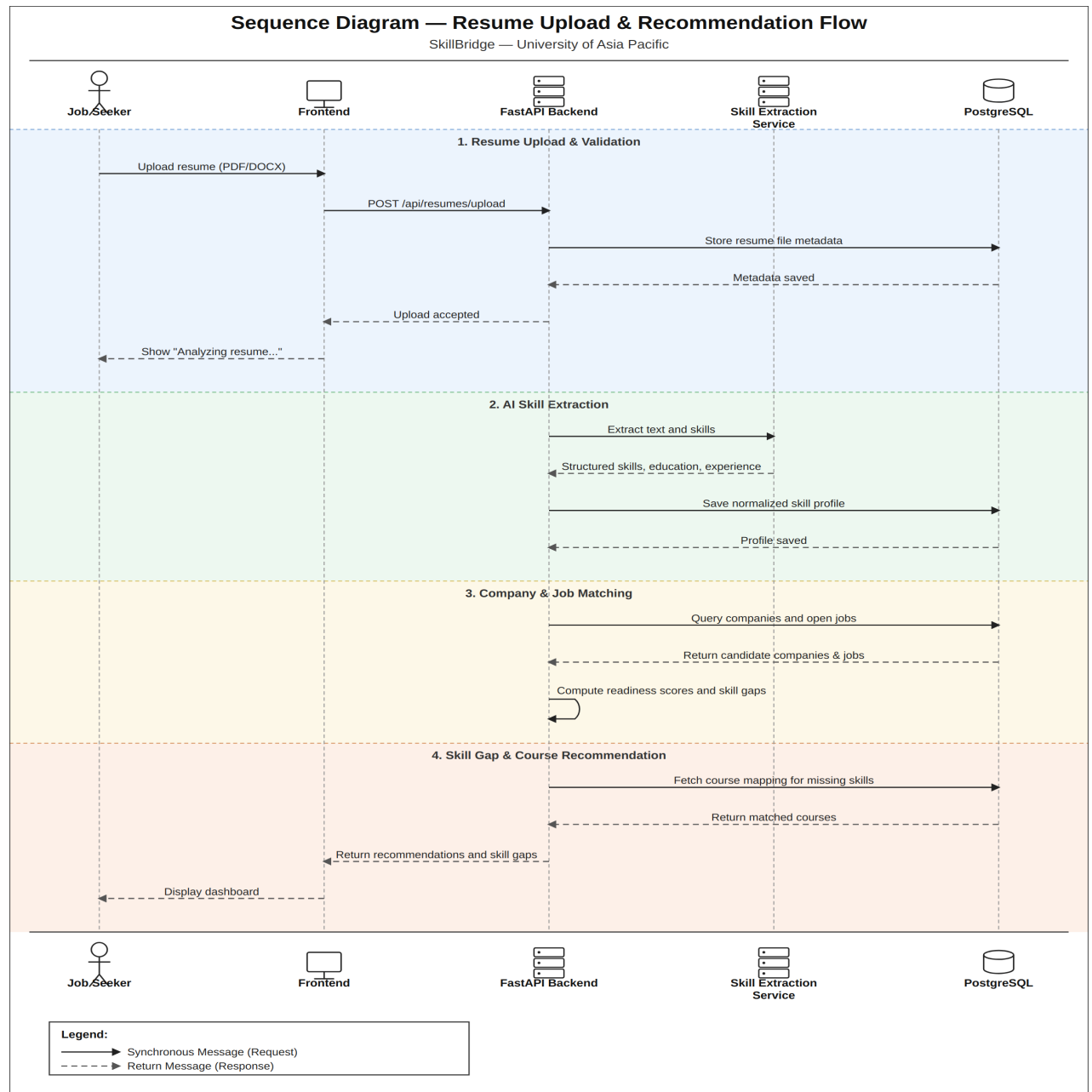


Figure 3.9.1: Sequence Diagram — Job Seeker

3.9.2. Recruiter / Employer Sequence Diagram

The sequence diagram below is specific to the Recruiter/Employer module: submitting a job posting (saved with status=pending_review), the Admin verification gate that must clear before it goes live (Section 3.3.14), searching consenting candidate profiles by skill-match score, and shortlisting a candidate to send an interest notification — distinct from the Job Seeker sequence in Section 3.1 and the Admin sequence in Section 3.3.14, though all three ultimately act on the same underlying schema.

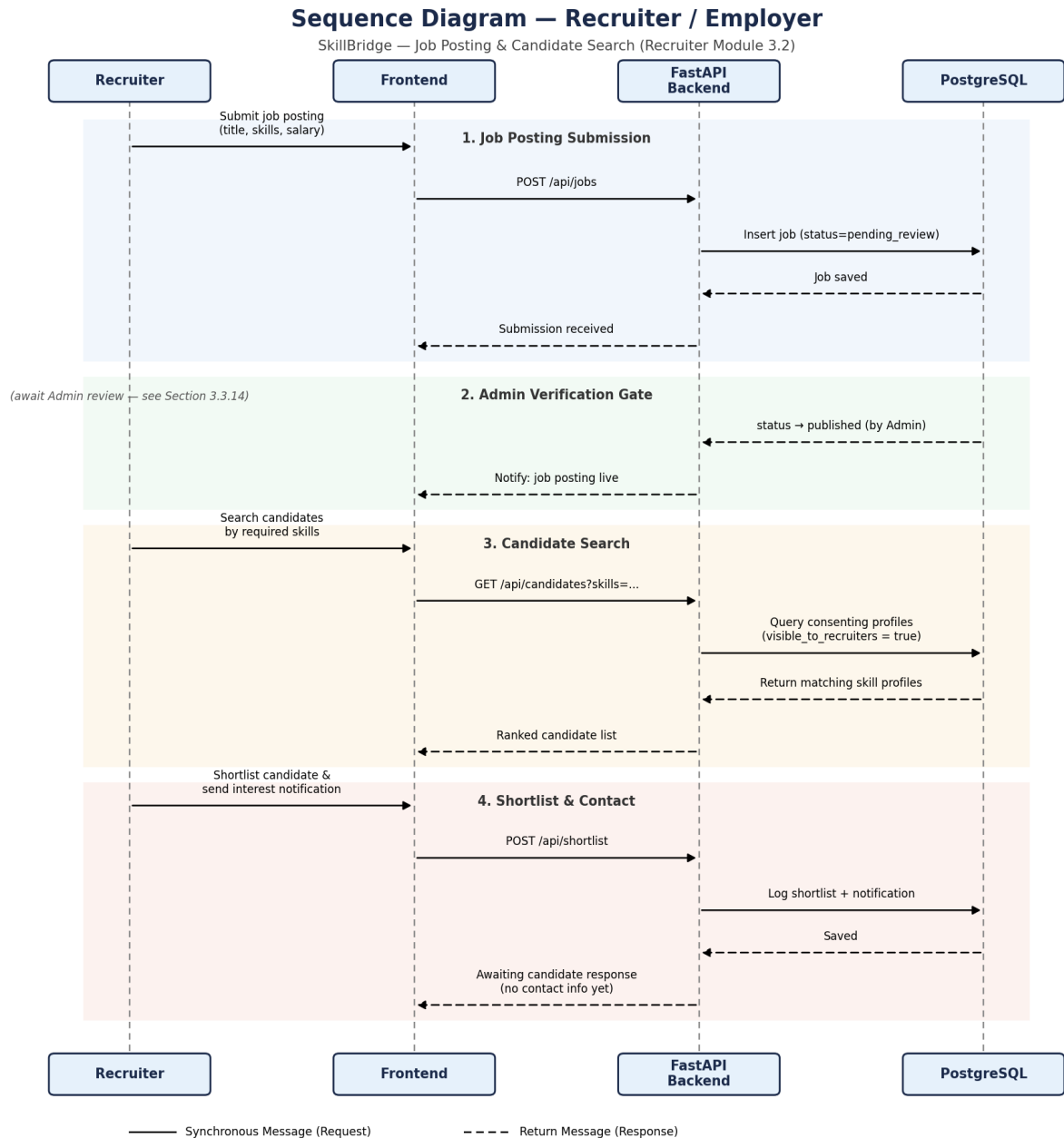


Figure 3.9.2: Sequence Diagram — Recruiter / Employer

3.9.3. Admin / Authority Sequence Diagram

The sequence diagram below is specific to the Admin module: opening the pending recruiter accounts queue and approving an account, then opening the pending job postings queue and publishing a posting — distinct from the Job Seeker sequence in Section 3.1 and the Recruiter sequence in Section 3.2.14, though all three ultimately act on the same underlying schema.

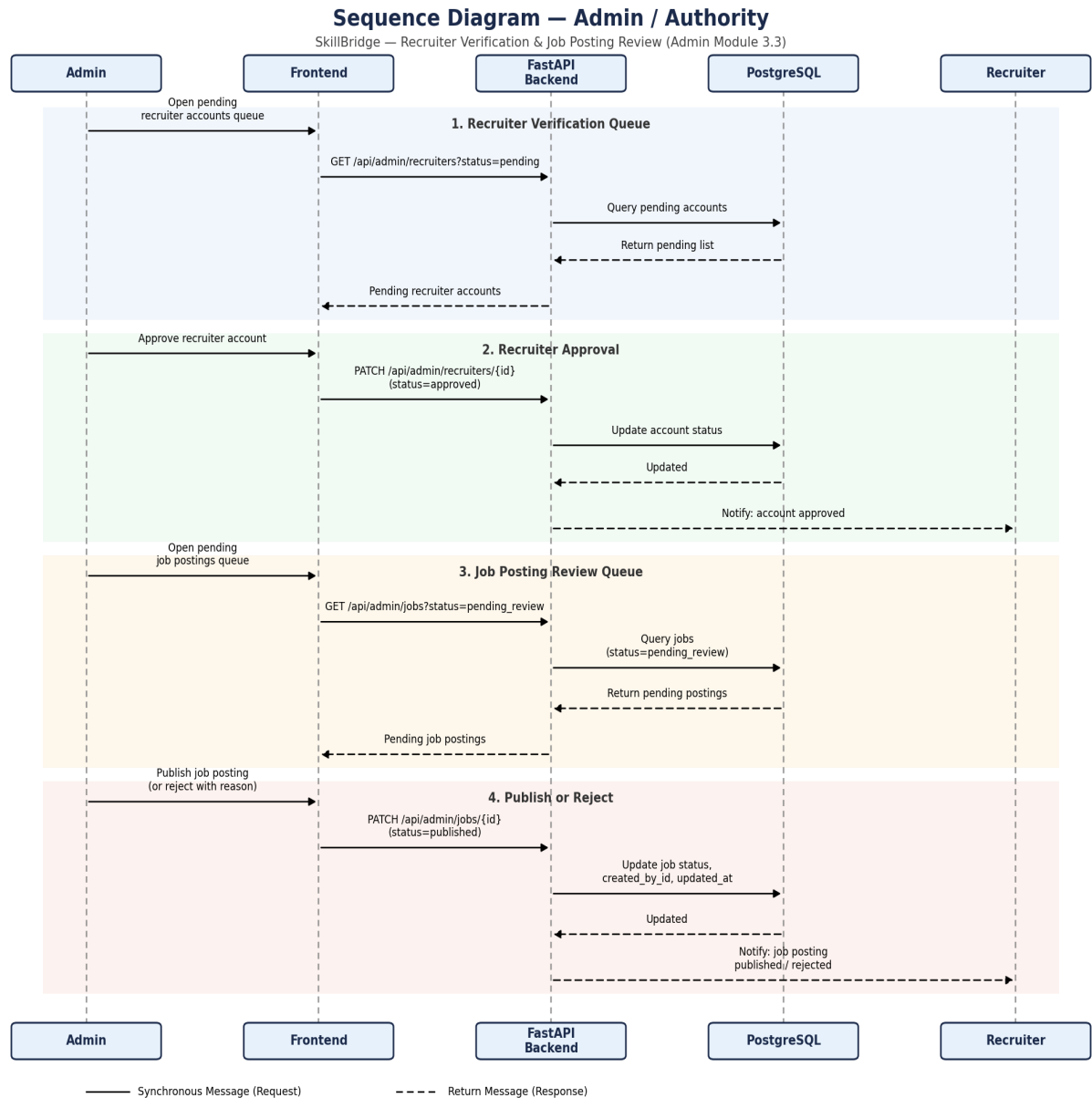


Figure 3.9.3: Sequence Diagram — Admin / Authority

3.10. State Diagram

3.10.1. Job Seeker State Diagram

The state diagram models a Job Seeker's profile as it moves from initial registration through resume processing to a state where recommendations are available. A failed parse routes to an explicit Flagged for Admin Manual Review state rather than a dead end; a successful re-upload transitions the profile back through processing; and the consent

toggle is modeled as a self-transition, since it changes recruiter visibility without otherwise affecting the profile's lifecycle stage.

State Diagram — Job Seeker

SkillBridge — Job Seeker Profile Lifecycle (Job Seeker Module 3.1)

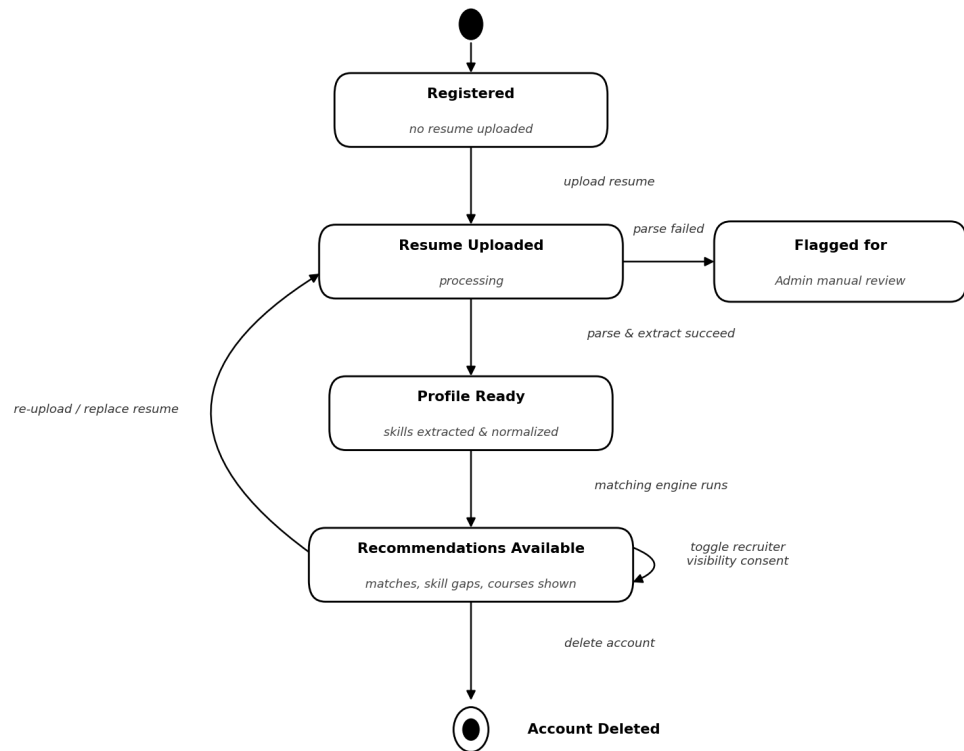


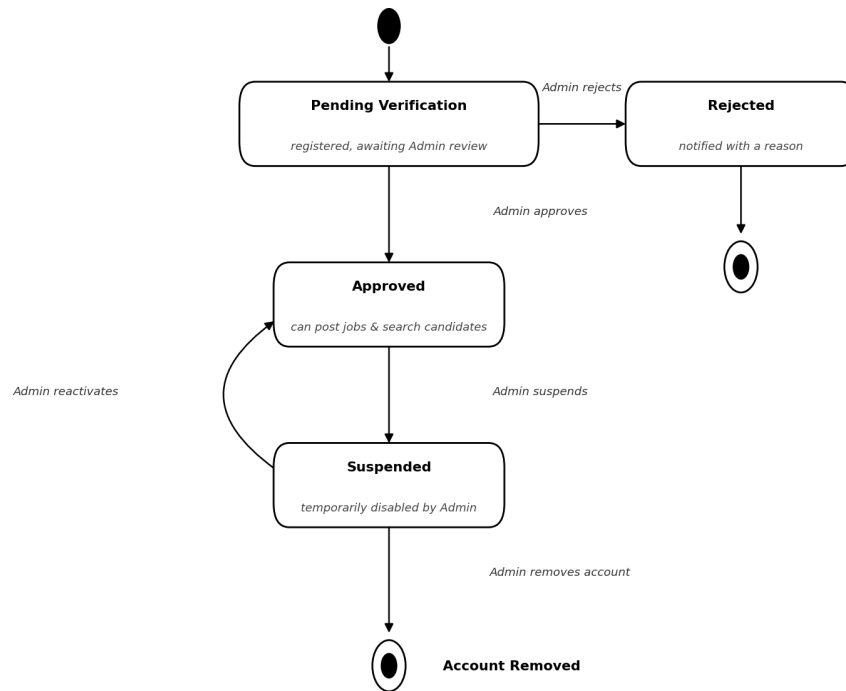
Figure 3.10.1: State Diagram — Job Seeker

3.10.2. Recruiter / Employer State Diagram

The state diagram below models a recruiter account's own lifecycle, distinct from the Job Seeker profile lifecycle in Section 3.1.11: from Pending Verification, an Admin decision branches the account to either Rejected (a terminal state, with a reason given) or Approved. An Approved account can be temporarily Suspended by the Admin and later reactivated, or permanently removed, which is also terminal.

State Diagram — Recruiter / Employer

SkillBridge — Recruiter Account Lifecycle (Recruiter Module 3.2)

*Figure 3.10.2: State Diagram — Recruiter / Employer***3.10.3. Recruiter / Employer State Diagram**

The state diagram below models a job-posting record's lifecycle under Admin oversight, distinct from the Job Seeker profile lifecycle (Section 3.1.11) and the Recruiter account lifecycle (Section 3.2.15): a Draft moves to Pending Review, where the Admin either rejects it (returned to the recruiter for edits and resubmission) or approves it to Published. A Published posting can be Closed normally (deadline reached or position filled) or Removed directly by the Admin for a policy violation, with both paths ending in an Archived record.

State Diagram — Admin / Authority

SkillBridge — Job Posting Record Lifecycle Under Admin Oversight (Admin Module 3.3)

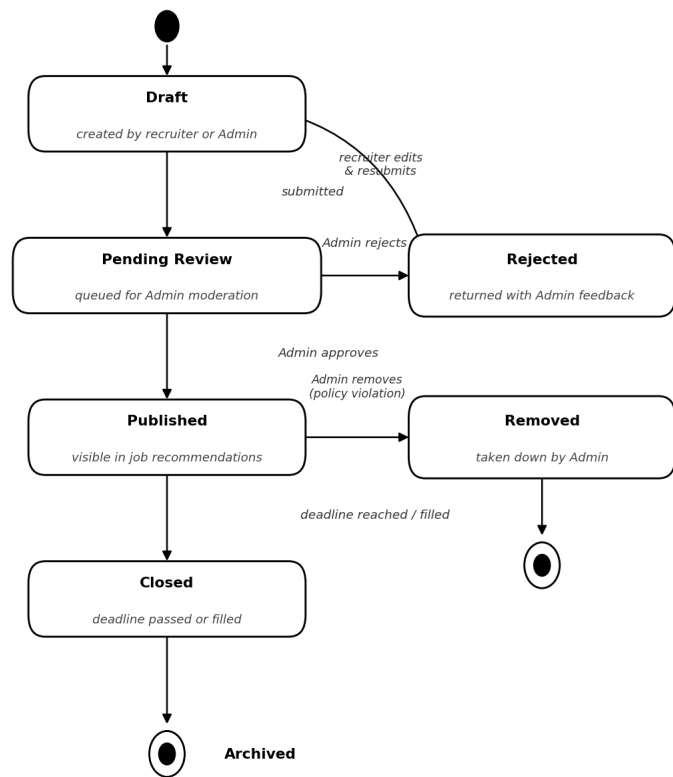


Figure 3.10.3 State Diagram — Admin / Authority

4. Architecture

4.1 Architectural Model/Style Used

For the SkillBridge platform, two architectural concepts are employed:

Client-Server Architecture with a Centralized Data Store: At the overall system level, a React frontend (client) communicates with the FastAPI backend (server) over HTTPS/REST, and a central PostgreSQL database resides at the center of the backend, accessed by every module — the Auth & User module, the Resume & Skill Extraction module, the Matching & Recommendation module, and the Admin & Recruiter module. Each module independently executes its own processes while reading from and writing to the shared central database, which keeps the modules loosely coupled: a new module (e.g. a future Messaging module) could be added later without modifying the others, since it would simply become another client of the same central store. The Resume & Skill Extraction module additionally calls an external Skill Extraction Service (backed by the DeepSeek API and, later, spaCy/Sentence Transformers) to turn raw resume text into structured skills — keeping that AI step a replaceable component rather than baking it into the core request/response cycle.

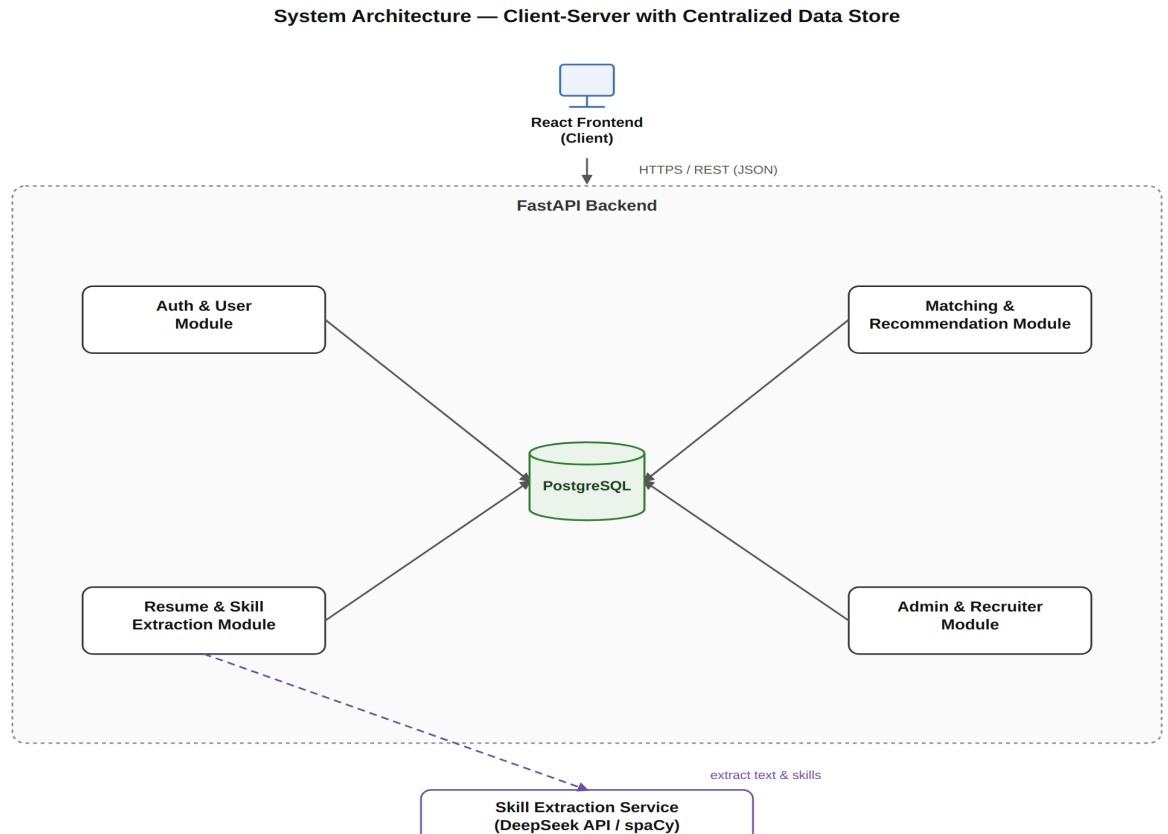
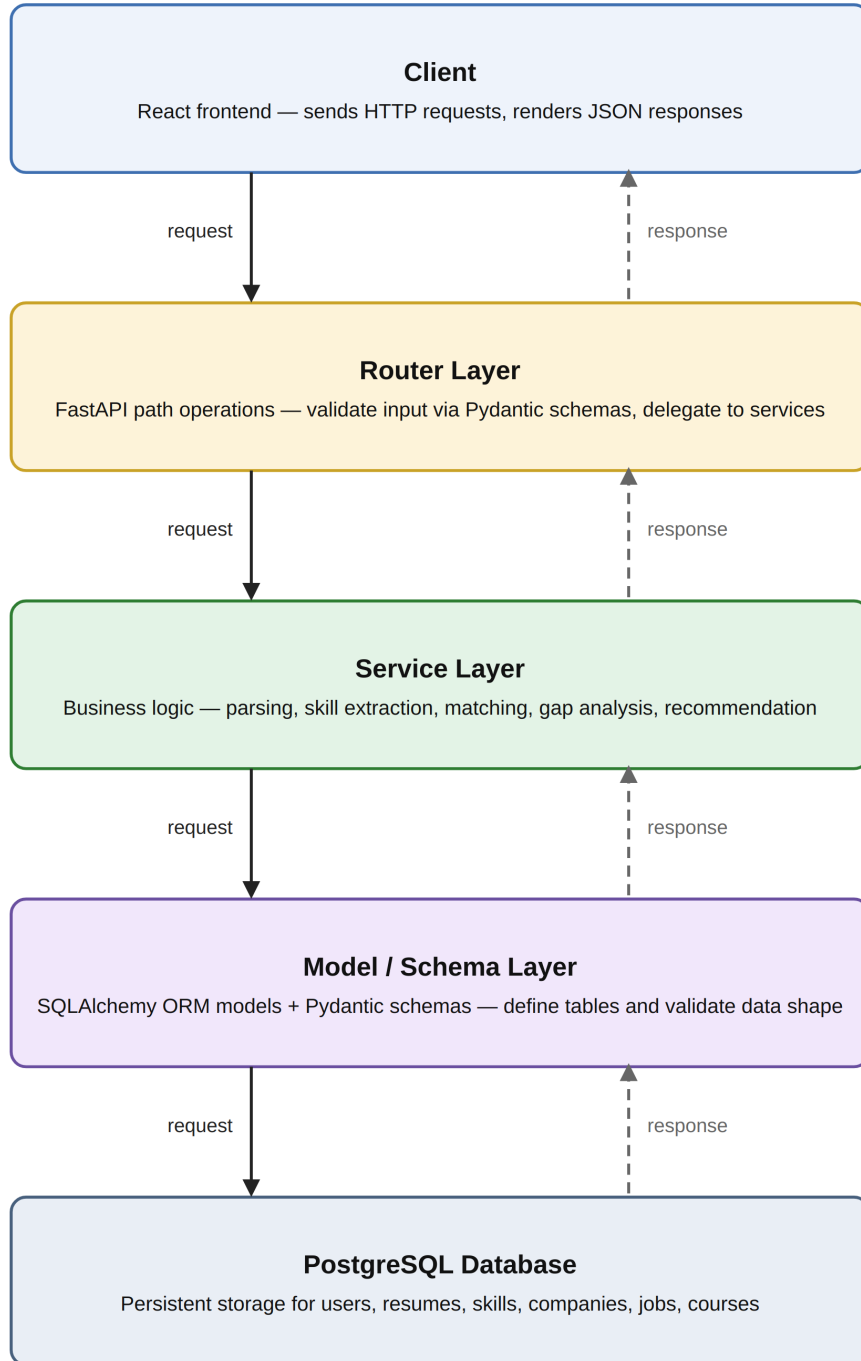


Figure 4.1.1.1: System Architecture — Client-Server with Centralized Data Store

Layered (Router–Service–Model) Architecture: At the application level, the FastAPI backend follows a layered pattern, which is FastAPI's natural analogue to the Model-View-Template (MVT) or Model-View-Controller (MVC) patterns used by other frameworks. The Router layer receives HTTP requests and validates them against Pydantic schemas. The Service layer contains the business logic — resume parsing, skill extraction, matching, skill-gap analysis, and course recommendation. The Model layer defines the database structure through the SQLAlchemy ORM, and Pydantic schemas separately validate and serialize the data that crosses the API boundary.

Layer	Role	SkillBridge Example
Router	Receives HTTP requests, validates input via Pydantic schemas, and delegates to the service layer.	POST /api/resumes/upload → calls ResumeService.upload_resume()
Service	Contains the business logic: parsing, skill extraction, matching, skill-gap analysis, and course recommendation.	SkillExtractionService.extract() → calls the AI/NLP layer and normalizes results
Model	Defines the database structure. Written as SQLAlchemy classes; the ORM maps them to tables.	Resume model → resumes table, Job model → jobs table
Schema	Validates and serializes request/response data using Pydantic, independent of the ORM models.	ResumeUploadResponse schema → validates the JSON returned to the frontend

*Table 4.1.1.1: Layered (Router–Service–Model) Breakdown for SkillBridge***Layered Architecture (Router → Service → Model)***Figure 4.1.1.2: Layered Architecture (Router → Service → Model)*

4.1.2. Rationale for Choosing Architectural Model/Style:

The SkillBridge system is fundamentally data-driven — every feature (resume uploads, extracted skill profiles, company and job records, skill-gap results, course mappings) revolves around CRUD (Create, Read, Update, Delete) operations on a central database. A client-server architecture with a centralized data store naturally fits this requirement, since every module accesses the same store independently, and it directly supports the integrability the platform needs as recruiter and admin features grow.

A full microservice architecture would be unnecessarily complex for the scope of this academic project. The client-server approach with a layered backend is simple, keeps the team's FastAPI code organized, and matches the nature of the project — nearly every user action (uploading a resume, viewing a recommendation, approving a recruiter, mapping a course to a skill) is essentially a database operation reached through a thin Router → Service → Model chain. The one component that is already logically separated — the Skill Extraction Service — is deliberately kept as a distinct service behind its own interface, consistent with the project's design principle of keeping AI extraction, the structured user profile, and the recommendation engine as separate, explainable components (Section 2.1); this also means it could be pulled out into its own deployable service later without restructuring the rest of the backend.

The layered pattern is a natural fit for FastAPI: it provides a built-in separation of concerns — request handling (Router), business logic (Service), and data structure (Model/Schema) — which promotes maintainability and clean code organization. For a student team, following this structure from the outset (already reflected in the app/routers, app/services, app/models, and app/schemas packages scaffolded for the project) avoids unnecessary complexity while keeping development structured as new parts are built.

4.2. Technology, Software, and Hardware Used:

- **FastAPI (Python):** FastAPI is a modern, high-performance Python web framework for building APIs. It provides automatic request validation via Pydantic, interactive Swagger/OpenAPI documentation, and native async support. FastAPI's structure naturally supports the layered Router–Service–Model pattern used for SkillBridge.
- **PostgreSQL:** PostgreSQL is a relational database management system known for reliability, strong data-integrity guarantees, and rich support for structured and semi-structured (JSON) data. SkillBridge uses PostgreSQL as the central data store for users, resumes, skill profiles, skills, companies, jobs, courses, and recommendations.
- **SQLAlchemy & Alembic:** SQLAlchemy is the ORM that maps Python classes to PostgreSQL tables and lets the service layer query the database without writing raw SQL. Alembic manages versioned schema migrations as the data model evolves.
- **HTML/CSS:** HTML (HyperText Markup Language) and CSS (Cascading Style Sheets) are used for the frontend (Template layer). HTML provides the structure of web pages, while CSS handles the presentation and styling. The combination enables responsive web design accessible across desktop and mobile devices.
- **pdfplumber & python-docx:** These Python libraries extract raw text from uploaded PDF and DOCX resumes respectively, feeding the skill-extraction step.

- **DeepSeek API (with spaCy and Sentence Transformers planned):** The initial AI/NLP layer used for structured skill, education, and experience extraction from resume text, with spaCy and Sentence Transformers planned for later semantic matching.
- **Visual Studio Code:** VS Code is used as the primary Integrated Development Environment (IDE) for code editing, debugging, and version-control integration.
- **GitHub:** GitHub is used for version control and repository hosting, enabling collaborative development, code tracking, and change management among team members.

5. Design

5.1 Component Level Design Following Pattern

A component is a higher-level interface that exports and wraps its own implementation — an interchangeable, reusable, portable, and modular unit of well-specified functions. A software component is a segment of a system that either communicates with other parts through a defined set of functions, or that encapsulates a specific piece of functionality on its own. What characterizes a component, in either case, is that it is a composition unit: it has internal composing parts, a contractually specified interface, and explicit dependencies on its surrounding context.

The SkillBridge platform is decomposed into four modules — Authentication, the Resume & Skill Matching Module, the Recruiter / Employer Module, and the Admin Panel — as shown in the structure chart below. The following subsections detail the component-level design for the three core feature modules.

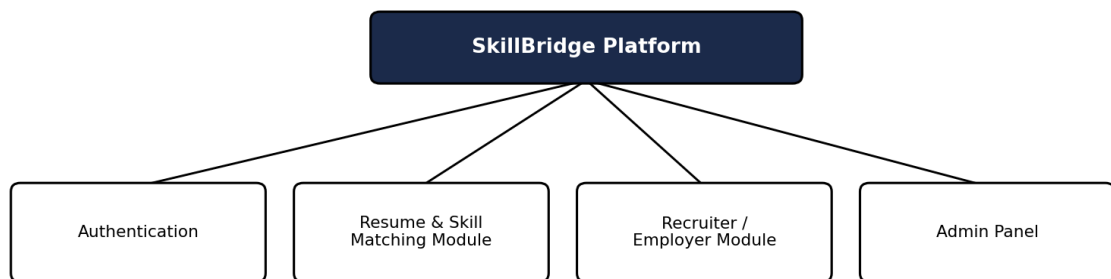


Figure 5.1: Module Structure Chart — SkillBridge

Resume & Skill Matching Module

The Resume & Skill Matching Module is the primary component of the SkillBridge platform. It enables a Job Seeker to upload a resume in PDF or DOCX format, which the system parses to extract raw text before running it through an AI/NLP skill-extraction step. Extracted skills are normalized against an Admin-curated taxonomy so that variants of the same skill are treated consistently. The resulting structured skill profile is then compared against the companies and open jobs maintained in the platform, producing ranked recommendations with a readiness score, an explainable skill-gap breakdown against a target role, and course recommendations mapped to each missing skill. The system tracks the full lifecycle: resume upload → parsing → skill extraction → normalization → matching → skill-gap analysis → course recommendation → dashboard display.

Recruiter / Employer Module

The Recruiter / Employer Module provides a bridge between employers and the platform's pool of job seekers. Recruiters register with a work email and company name and undergo Admin verification before their account is activated. Once approved, a recruiter can submit and manage job postings (title, required skills, experience, salary range), which similarly pass through an Admin review gate before becoming publicly visible. Recruiters can search and filter candidate skill profiles by skill-match score, but only among job seekers who have explicitly opted in to recruiter visibility. A recruiter may shortlist a

candidate and send an interest notification; contact details are exchanged only after the candidate accepts, preserving the job seeker's privacy until that point.

Admin Panel

The Admin Panel is the central moderation and management component. It provides administrators with tools to review and approve or reject pending recruiter registrations, review and publish or reject submitted job postings, curate the master skill taxonomy and the skill-to-course mapping used for recommendations, manage company records, and monitor platform-wide statistics (total users, resumes processed, job postings, recruiter accounts). The Admin Panel ensures data quality and policy compliance across both the Job Seeker and Recruiter / Employer modules.

5.2 Dataflow Diagram

The first-level Data Flow Diagram (DFD) for the SkillBridge platform encapsulates the core processes essential for its operation. The User Management & Authentication process serves as the entry point, handling Job Seeker and Recruiter registration and Admin login, and ensuring secure storage of account records within the system. The Resume Processing process extracts and normalizes a Job Seeker's skills from an uploaded resume. The Matching & Recommendation process compares that skill profile against job and company data to produce ranked matches, skill-gap results, and course recommendations. The Job Posting Management process handles a recruiter's submissions through to Admin publication, the Candidate Search process lets recruiters query consenting candidate profiles, and the Admin Moderation process governs recruiter verification and job-posting approval across the platform.

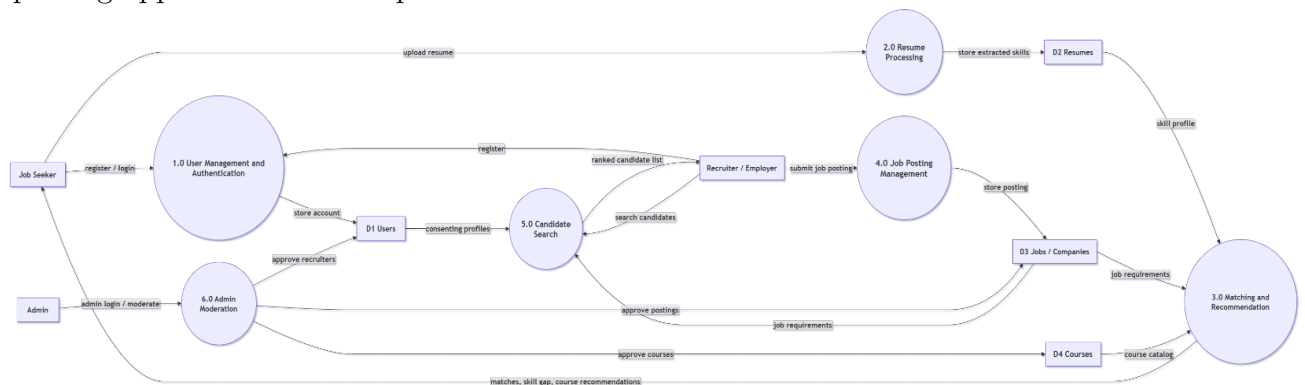


Figure 5.2: Level-0 Data Flow Diagram — SkillBridge

Job Seeker DFD

In the Job Seeker Data Flow Diagram, a job seeker begins with registration (name, email, password), after which the Register / Login process creates an account record. Once logged in, the job seeker uploads a resume, which the Upload & Analyze Resume process parses into a structured, normalized skill profile stored against the resume record. The job seeker then accesses two key views: View Matches & Skill Gap, which returns ranked company and job matches, a readiness score, and a skill-gap breakdown drawn from the stored job and company data, and View Course Recommendations, which maps each missing skill to a relevant course from the course catalog. This job-seeker-centric

DFD reflects the core value loop of the platform, from a raw resume to an actionable, explainable set of recommendations.

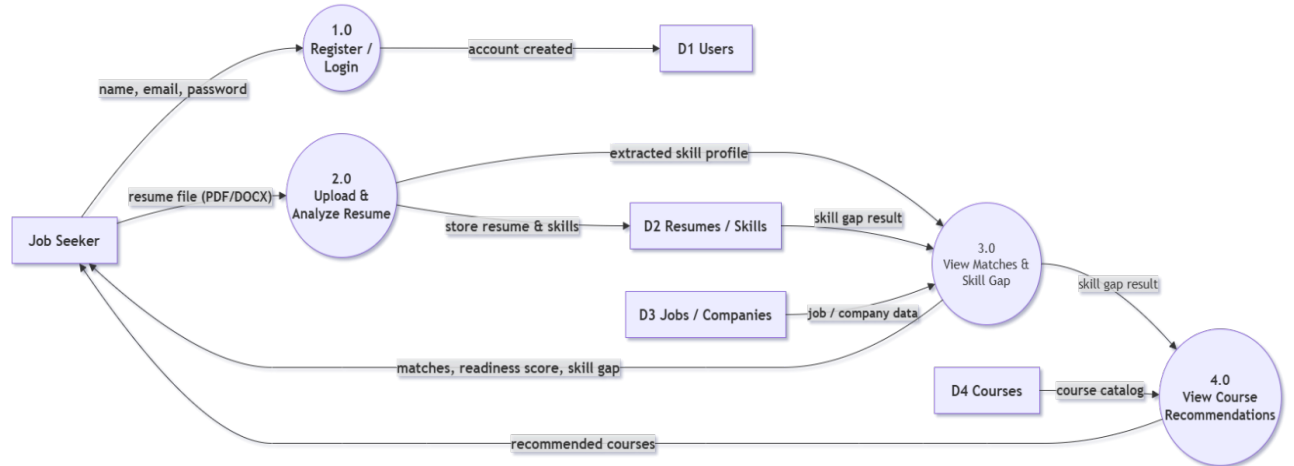


Figure 5.3: Job Seeker Data Flow Diagram — SkillBridge

Recruiter / Employer DFD

In the Recruiter / Employer Data Flow Diagram, a recruiter begins with registration (work email, company name), after which the Register / Login process creates an account record pending Admin verification. Once approved, the recruiter accesses two key processes: Submit Job Posting, which stores a new posting against the Jobs / Companies store with a pending_review status until the Admin publishes it, and Search Candidates, which queries the Resumes / Skills store for consenting profiles matching the posting's required skills and returns a ranked candidate list. A recruiter can then act on that list through Shortlist & Notify, which logs the shortlist entry and sends an interest notification — the candidate's contact details are withheld until they accept.

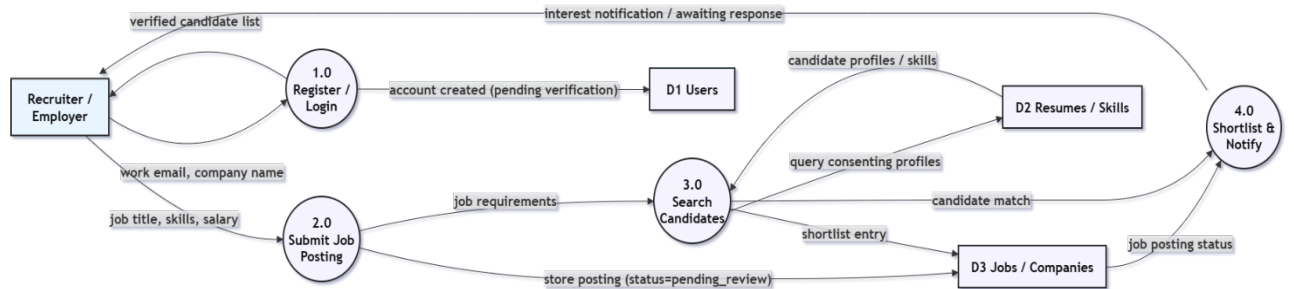


Figure 5.4: Recruiter / Employer Data Flow Diagram — SkillBridge

Admin / Authority DFD

In the Admin / Authority Data Flow Diagram, the Admin governs three moderation processes and one reporting process. Review Recruiter Verification approves or rejects pending recruiter accounts in the Users store. Review Job Postings approves, publishes, or rejects postings held in the Jobs / Companies store. Manage Skill Taxonomy lets the Admin add or edit entries in the Skills store and the skill-to-course mapping in the Courses store, which together drive every Job Seeker's skill-gap and course recommendations. Monitor Platform Statistics reads aggregate counts back from the Users and Jobs / Companies stores and skill-demand data from the Skills store, and returns a consolidated statistics dashboard to the Admin.

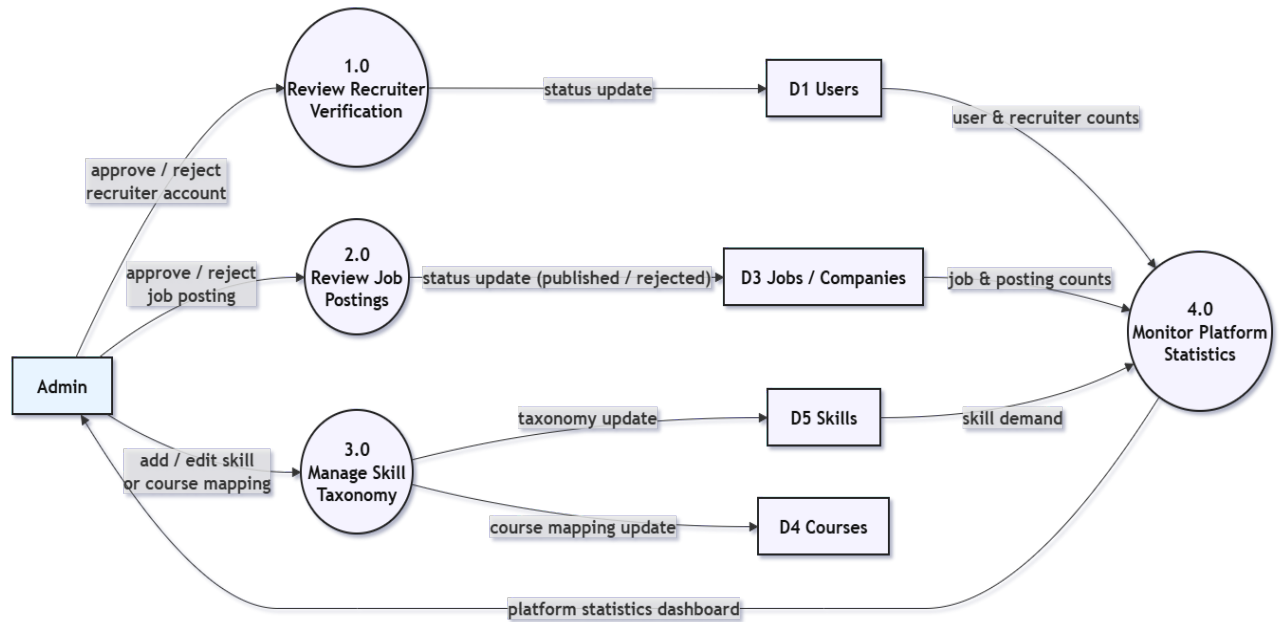


Figure 5.5: Admin / Authority Data Flow Diagram — SkillBridge

5.2.1 Entity Relationship Diagram

An Entity-Relationship Diagram (ERD) is a visual representation of the data model that depicts the entities within a system, the relationships between those entities, and the cardinality constraints that govern how records in one table relate to records in another. Below is the current ERD for SkillBridge, generated directly from the implemented Supabase/PostgreSQL schema.

The SkillBridge ERD consists of 8 entities and 9 relationship types that together represent the complete data model of the platform. The entities cover the Job Seeker module (Resumes, ResumeSkills), the Recruiter / Employer module (Companies, Jobs, JobSkills), and shared reference data (Skills, Courses), all anchored by the Users entity. The relationships capture the platform's core data dependencies — uploads, extracted_from, tagged_with, posts, requires, maps_to, and teaches — together with the created_by / updated_by audit trail that every non-Users table carries back to Users. A separate alembic_version table, visible in the diagram, is Alembic's own schema-migration bookkeeping table rather than a business entity, and is not part of the domain model described above.

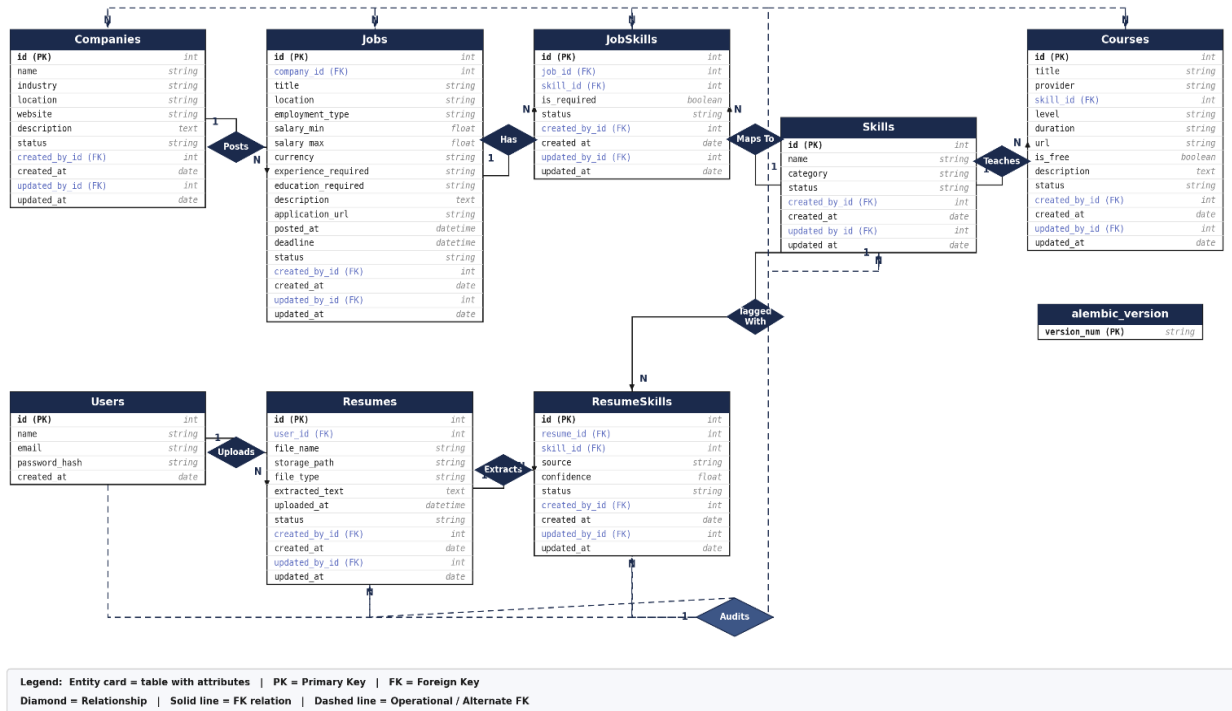


Figure: Attribute-Based Entity Relationship Diagram — 9 entities with keys, attributes, and 8 relationships for SkillBridge Platform

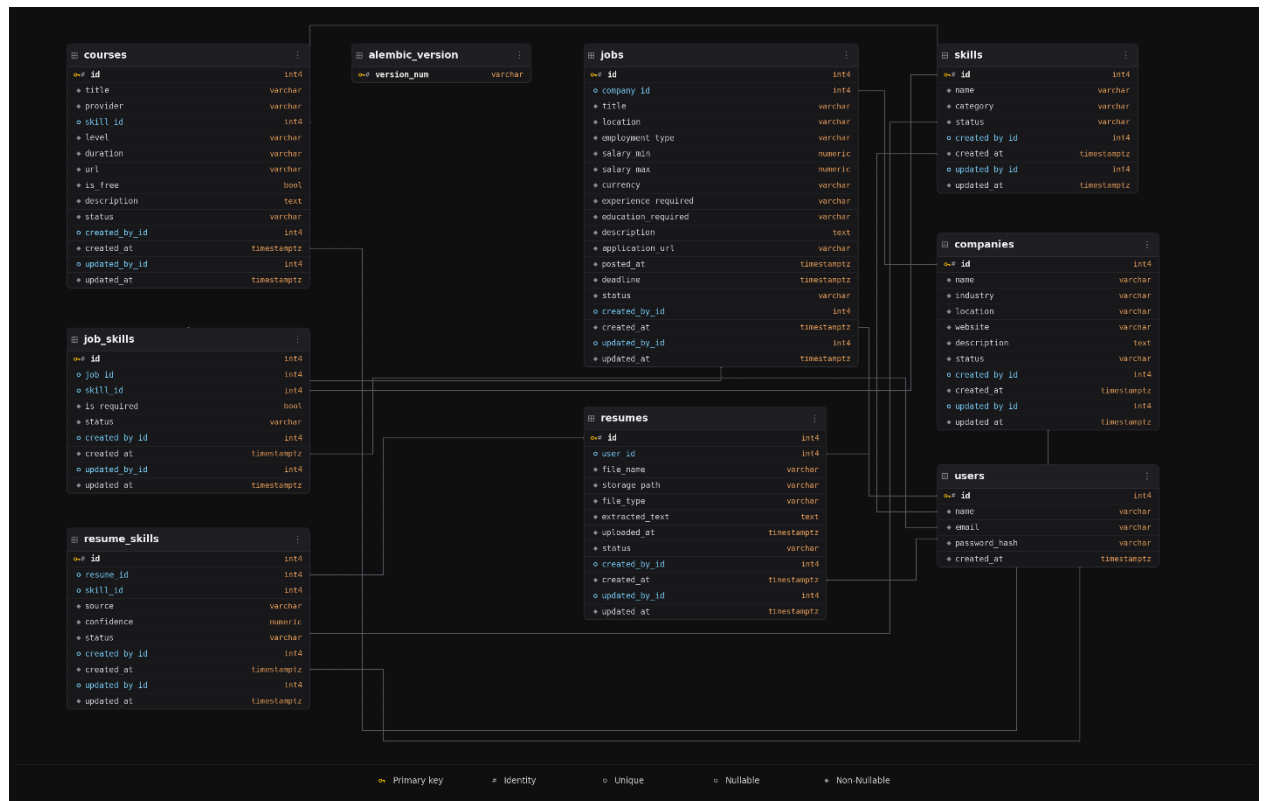


Figure 5.6: Entity Relationship Diagram — SkillBridge